MSH chain notebooks

Function reference

Magnet–superconductor-hybrid chain: tensor networks (TensorBinding.jl) and dense exact diagonalization

Notebook	Content
`msh_chain_tn.ipynb`	Julia, TensorBinding.jl / ITensors: MPO, KPM, DMRG, TDVP
`msh_chain_python_ed.ipynb`	Python / NumPy: dense BdG exact diagonalization
`msh_chain_compare.ipynb`	Python: row-by-row comparison of the two result files

Repository: `MSH_Tensor_Network_Methods`

2026-10-29

Contents

1 Overview **3**

1.1 The model . . . 3

1.2 Nambu conventions of the two codes . . . 4

1.3 Workflow and data files . . . 4

1.4 Mapping between the implementations . . . 5

I Tensor-network notebook `msh_chain_tn.ipynb` **6**

2 Building blocks (TN §2) **6**

3 The Hamiltonian builder (TN §3) **7**

4 Dense helpers and validation (TN §4–5) **7**

5 LDOS and the spin-ramp definitions (TN §6) **8**

5.1 Spin ramp: `smoothstep`, `SpinRamp` and its methods . . . 9

6 LDOS with TensorBinding's KPM routine (TN §7) **9**

7 Majorana zero modes and the fermion mode (TN §8) **12**

7.1 Small helpers . . . 13

7.2 The pipeline . . . 13

7.3 §8a example: chain ends and domain walls . . . 14

8 Charge difference between the logical states (TN §9) **15**

8.1 §9a: ΔQ versus segment length . . . 16

9 Time-dependent ramp (TN §10) **17**

9.1 §10a: the ramp on 64 sites . . . 19

10 How to adapt the notebook (TN §11) **20**

11 Saving results (TN, last section) **21**

II Dense Python notebook `msh_chain_python_ed.ipynb` **22**

12 Model (Python §0) **22**

13 Hamiltonian check and spectrum sweep (Python §1–2) **23**

14 Lorentzian LDOS (Python §3) **23**

15 Jackson-KPM LDOS (Python §4) **23**

16 Majorana modes (Python §5) **24**

17 Charge difference vs Majorana separation (Python §6) **26**

18 Spin ramp (Python §7) **27**

19 Interactive views and saving (Python §7b, last cell) **28**

III Comparison notebook `msh_chain_compare.ipynb` **29**

20 Method **29**

21 What each block compares **30**

22 Results **30**

23 TensorBinding routines used (from the v0.2.0 source) **32**

1 Overview

This document describes every function and struct defined in the three MSH-chain notebooks: what each computes, the method, its inputs and outputs, the numerical settings that matter, and where it is used. All statements are taken from the notebook code, markdown and printed outputs. Numbers quoted from outputs are for the 64-site chain ($L = 6$) that all three notebooks use. Energies and times are in units of the hopping t (with $\hbar = 1$).

1.1 The model

A single chain of N sites (open boundaries) with a classical spin texture $\mathbf{S}_n$ on an s-wave superconductor. The particle block is

$$H_0 = \sum_n c_n^\dagger(-\mu + J\,\mathbf{S}_n \cdot \sigma)c_n + \sum_n \Big[c_{n+1}^\dagger(-t + i\alpha\,\sigma_y)c_n + \text{h.c.}\Big],$$

with on-site singlet pairing $\Delta\,(c_\uparrow^\dagger c_\downarrow^\dagger - c_\downarrow^\dagger c_\uparrow^\dagger)$. Here $c_n = (c_{n\uparrow}, c_{n\downarrow})^T$, μ is the chemical potential, J the exchange coupling, α the Rashba coupling and σ the Pauli matrices in spin space. The TN notebook calls this "the 1D slice of the kwant CZ model (`chain` onsite + `hop_x` hopping)".

Table 1: Model parameters (defaults in both codes; "CZ-project values").

Symbol	Julia keyword	Python key	Value
t	`t`	`t`	1.0
μ	`μ`	`mu`	-7.0 (fixed throughout)
α	`α`	`alpha`	0.9
Δ	`Δ`	`Delta`	2.4
J	`J`	`J`	5.8
φ	`φ`	`phi`	$\pi/2$

Spin texture. The spins are

$$\mathbf{S}_n = (\cos\varphi \sin\theta_n,\ \sin\varphi \sin\theta_n,\ \cos\theta_n), \qquad \theta_n = (-1)^n\,\tfrac{\pi}{2} f(n),$$

with n the 0-indexed site and $f(n) \in [0, 1]$ the *rotation fraction*. $f = 0$ puts all spins along $+\hat{z}$ (ferromagnetic, out of plane), which is topological at $\mu = -7$ with Majorana end modes. $f = 1$ gives alternating $\pm\hat{y}$ (antiferromagnetic, in plane), which is trivially gapped. The chemical potential is never changed; the phase is switched only by rotating the spins through f. For spins along $\hat{z}$ the TN notebook gives the 1D topological window $\Delta^2 + (\mu + 2t)^2 < J^2 < \Delta^2 + (\mu - 2t)^2$ (Rashba does not shift these bounds); with the defaults this reads $30.76 < 33.64 < 86.76$.

1.2 Nambu conventions of the two codes

Table 2: Basis conventions. Only basis-independent quantities (spectra, spin-summed LDOS and densities, ΔQ) are compared between the codes.

	Julia (`msh_chain_tn`)	Python (`msh_chain_python_ed`)
Nambu spinor	$(c_\uparrow, c_\downarrow, c_\uparrow^\dagger, c_\downarrow^\dagger)$	$(c_\uparrow, c_\downarrow, c_\downarrow^\dagger, -c_\uparrow^\dagger)$, i.e. $\psi^\dagger = (c_\uparrow^\dagger, c_\downarrow^\dagger, c_\downarrow, -c_\uparrow)$
BdG form	$\begin{pmatrix} H_0 & \Delta\, i\sigma_y \\ (\Delta\, i\sigma_y)^\dagger & -H_0^* \end{pmatrix}$	$-\mu\tau_z + \Delta\tau_x + J\,\mathbf{S}\cdot\sigma$ on site, hopping $H_{i+1,i} = -t\tau_z - i\alpha\tau_z\sigma_y$
Dense index	nambu-major: $(\text{nambu}-1)\,2N + (\text{spin}-1)\,N + n$, i.e. `kron(nambu, spin, position)`	site-major: `4*x + c`, `c` $= 0 \ldots 3$ (“`ltslts`”, “`udd-u`”)
Particle–hole	$C = \tau_x K$: $\tau_x H^* \tau_x = -H$	$C = \tau_y\sigma_y K$: $H = -\tau_y\sigma_y H^* \tau_y\sigma_y$
Rashba sign	as in H_0 above	opposite ($-t - i\alpha\sigma_y$ on $H_{i+1,i}$); the ramp is also run with $\alpha = -0.9$ to reproduce the Julia model
MPO sites	`[nambu; spin; pos₁ … pos_L]`, $N = 2^L$, `pos₁` = most significant bit	—

The Rashba sign does not change static quantities (spectrum, LDOS, Majoranas, $\Delta Q(\ell)$), but it changes the dynamics of the spin ramp (Section 18).

1.3 Workflow and data files

1. Run `msh_chain_tn.ipynb`. Sections 7a, 9a and 10a load cached results from `msh_chain_ldos_kpm_ramp.jls`, `msh_chain_charge.jls` and `msh_chain_ramp.jls` unless the flags `RUN_FULL_KPM`, `RUN_CHARGE`, `RUN_RAMP` are set to `true`. First runs take about 30, 10 and 4 minutes; sections 1–8 take a few minutes. The last cell writes 43 arrays to `msh_chain_tn_results.npz`.
2. Run `msh_chain_python_ed.ipynb` (about 4 minutes). It writes `msh_compare_python.npz` (42 arrays). The sliders of section 7b need a live kernel.
3. Run `msh_chain_compare.ipynb` (seconds). It loads both `.npz` files and prints one line per comparison.

The `.jls` caches are tied to the function and struct definitions of the TN notebook. After changing `SpinRamp` or the diagnostics, the matching `RUN_*` flag must be set once to regenerate them.

1.4 Mapping between the implementations

Table 3: Where the two notebooks compute the same quantity, and the comparison row that checks it.

Quantity	TN notebook (Julia)	Python notebook	Compare row(s)
Spin texture	`spin_texture`	`spin_texture`	(all)
BdG Hamiltonian	`msh_chain` (MPO), `reference` (dense)	`calc_msh_ham`, `msh_ham`; `reference_julia` (port of `reference`)	(all; checked in-notebook)
$\|E\|_1, \|E\|_3$ vs f	§5 sweep with `dense`	§2 sweep	f sweep
Lorentzian LDOS	`ldos`	`particle_hole_weights`, `lorentz_ldos`	Lorentzian LDOS ×4
Ramp snapshots	`SpinRamp`, `f_nt`, `duration`, `wall_L`, `ldos_snapshots`	`ramp_f`, `ramp_duration`, `ramp_wall`, `cases_for`	(inputs)
KPM LDOS	`msh_tb` + `get_ldos_spatial`; `kpm_exact`	`kpm_scale`, `jackson`, `kpm_weight_matrix`, `kpm_ldos`	kpm_exact ×4; get_ldos_spatial ×4
Majoranas, ψ_f	`majorana_fermion` (`zero_mode_subspace`, `refine_subspace`, `fermion_from_subspace`), `uv_profile`	`majorana_fermion` (`individual_MZMs`, `takagi`, `fix_unitary`, `PHconj`)	centers, profiles, E_f (×2)
ΔQ, δn of a mode	`ΔQ_mode`, `δn_mode`	`dQ_mode`, `dn_mode`	(used below)
$\Delta Q(\ell)$	`majorana_pair_charge` (MPS), `exact_pair_charge` (dense)	`pair_charge`, `charge_density`	exact ΔQ (×2), FH, $E_f(\ell)$, δn, MPS ΔQ
Segment texture	`segment`, `f_segment`	`segment`, `f_segment`	(inputs)
Spin ramp	`RampHamiltonian`, `run_spin_ramp` (TDVP), `run_spin_ramp_exact`, `final_branches`, `adiabatic_reference`	`run_spin_ramp` (full BdG unitary, masks)	ramp rows (×7)

Part I

Tensor-network notebook `msh_chain_tn.ipynb`

The notebook loads `LinearAlgebra`, `ITensors`, `ITensorMPS`, `Printf`, `Plots` and `TensorBinding` (aliased `TB`). It collects the earlier scripts `msh_chain.jl`, `msh_chain_ldos.jl`, `msh_chain_majorana.jl`, `msh_chain_charge.jl` and `msh_chain_ramp.jl`. Every tensor-network result is checked against exact diagonalization or exact time evolution of the same model.

Why the BdG is assembled by hand. The notebook explains two problems with TensorBinding's high-level builders. `add_superconductivity!` builds the hole block as $\tau_z \otimes H_0$, which is wrong whenever H_0 has imaginary parts (the σ_y exchange component); the correct block is $-H_0^*$. `add_soc!(:rashba)` uses periodic shift operators and so adds a bond between the chain ends. Both break the comparison with the kwant model, so `msh_chain` uses TensorBinding's low-level pieces.

2 Building blocks (TN §2)

`σ0, σx, σy, σz`

Constant `ComplexF64` 2×2 Pauli matrices. They are passed explicitly as spin operators because TensorBinding's `:Sx/:Sy/:Sz` are $\sigma/2$, not σ.

`spin_texture(f; φ=π/2)`

```
spin_texture(f; φ=π/2) = n -> (θ = (-1)^n * (π/2) * f(n);
                               (cos(φ)*sin(θ), sin(φ)*sin(θ), cos(θ)))
```

Returns a closure `n -> (Sx, Sy, Sz)` for the 0-indexed site `n`, implementing $\mathbf{S}_n$ above. `f` is any function of the site with values in $[0, 1]$: uniform rotations (`n -> 0.3`), domains (`n -> 16 ≤ n < 48 ? 0.0 : 1.0`), or, for a time-dependent protocol, `n -> f(n, t)`. Used by `msh_chain`, `reference` and `RampHamiltonian`.

`diag_mpo_exact(v, pos; cutoff=1e-14)`

Builds the diagonal MPO $\mathrm{diag}(v)$ on the position qubits `pos` from a length-2^L vector `v`.

```
A = permutedims(reshape(ComplexF64.(v), ntuple(_ -> 2, L)), L:-1:1)
ψ = MPS(A, pos; cutoff=cutoff)
...
δ = ITensor(ComplexF64, s, s', dag(s''))   # δ(s, s', s'') copies the amplitude onto the
 ↪  diagonal
W[k] = ψ[k] * δ;  W[k] = replaceprime(W[k], 2 => 0; tags=tags(s))
```

Method. The vector is reshaped into an L-index tensor; the axes are reversed so that `pos[1]` is the most significant bit (Julia arrays are column-major). `MPS(A, pos)` compresses it by an exact tensor-train SVD with truncation `cutoff`. Each MPS tensor $\psi_s^{[k]}$ is then contracted with a copy tensor $\delta_{ss's''}$ ($= 1$ only if all three indices are equal), giving MPO tensors $W_{ss'}^{[k]} = \psi_s^{[k]}\,\delta_{ss'}$.

Why. It is used for the site-dependent exchange field. TensorBinding's `add_onsite!` / `get_diagonal_mpo` use quantics TCI, which "can silently drop sites of narrow or step-like profiles (for example sharp domain walls) at $L \geq 8$". The exact TT-SVD never does, at the cost of an $O(2^L)$ vector, which the notebook calls fine up to $L \approx 20$. Used in `msh_chain` and `RampHamiltonian`.

3 The Hamiltonian builder (TN §3)

`msh_chain(L, f; t=1.0, μ=-7.0, α=0.9, Δ=2.4, J=5.8, φ=π/2, cutoff=1e-12)`

```
"""
    msh_chain(L, f; t, μ, α, Δ, J, φ) -> (H_BdG::MPO, sites)

Sites are [nambu; spin; pos...].  `f(n)` (n = 0 … 2^L−1) is the rotation
fraction of the CZ protocol: 0 = topological (S ∥ ẑ), 1 = trivial (AFM ∥ ±ŷ).
"""
```

Inputs. `L`: number of position qubits ($N = 2^L$ sites). `f`: rotation profile. Keywords: model parameters (CZ-project defaults) and the MPO-addition `cutoff`. *Output.* The BdG MPO `HB` and its site indices `[ns; ss; pos]`.

Steps.

1. `TB.get_Hamiltonian("chain_1d", -t; L=L)` gives the hopping $-t$; `TB.add_onsite!(H, -μ)` adds the constant chemical potential.
2. `TB.add_spin!` adds the spin site (spin-degenerate so far).
3. Rashba: $\alpha\, i\sigma_y \otimes (K_u - K_d)$ with open-boundary shift operators `TB.shift_pair_mpos(pos, 1; cyclic=false)`; K_u is the forward shift $c^\dagger_{n+1} c_n$. Attached to the spin site with `TB.prepend_spin(·, ss, :iSy)`.
4. Exchange $J\,\mathbf{S}_n\cdot\sigma$: for each component $k \in \{x, y, z\}$ the vector $v_n = J\,S^k_n$ becomes an exact diagonal MPO (`diag_mpo_exact`) attached with the full Pauli matrix. Components with $\max|v| \le 10^{-14}$ are skipped.
5. BdG assembly with the Nambu site:

$$H_{\mathrm{BdG}} = P_p \otimes H_0 + P_h \otimes (-H_0^*) + \tau_+ \otimes \Delta\, i\sigma_y + \tau_- \otimes (\Delta\, i\sigma_y)^\dagger.$$

   On a plain MPO, `dag` is the elementwise complex conjugate, so `-1.0*dag(H0)` is $-H_0^*$; the pairing adjoint is `swapprime(dag(Hp), 0, 1)`.

The MPO bond dimension stays small (5–8 for the textures used), so the construction scales to large L. Example output: `msh_chain(6, n -> 0.0)` has 8 sites and maximal bond dimension 5. Used by `ldos`, `msh_tb`, the Majorana and charge sections, and (with $J = 0$) by `RampHamiltonian`.

4 Dense helpers and validation (TN §4–5)

These are for small systems only (dense dimension $4N$; practical up to $L \approx 7$).

`dense(mpo, sites)`

Contracts the MPO into a $D \times D$ matrix, $D = \prod \dim(\text{sites})$, ordered `kron(nambu, spin, position)` (big-endian over `sites`):

```
T = ITensors.@disable_warn_order prod(mpo)
return reshape(Array(T, reverse(prime.(sites))..., reverse(sites)...), D, D)
```

The index order is reversed because Julia arrays are column-major. The notebook notes it is much faster than `TB.get_matrix`, which applies the MPO to every basis state.

`reference(L, f; t=1.0, μ=-7.0, α=0.9, Δ=2.4, J=5.8, φ=π/2)`

Builds the same BdG matrix directly from the model definition, without MPOs, as an independent check. With $T_{n+1,n} = 1$ (open BC),

$$H_0 = \sigma_0 \otimes (-\mu\, \mathbb{1}_N) + \sum_n J\,(\mathbf{S}_n \cdot \sigma) \otimes P_n + [(-t\sigma_0 + i\alpha\sigma_y) \otimes T + \text{h.c.}], \quad H = \begin{pmatrix} H_0 & D \\ D^\dagger & -H_0^* \end{pmatrix}, \; D = \Delta\, i\sigma_y \otimes \mathbb{1}_N.$$

Used by the validation cell, `exact_pair_charge` and `dense_H`. The Python notebook ports it as `reference_julia`.

Validation cell (§4). For three textures on $L = 6$, the cell compares `dense(msh_chain(...))` with `reference`, checks particle–hole symmetry $CH^*C = -H$ with $C = \tau_x$ (`kron(σx, kron(σ0, I(N)))`), and prints the two smallest $|E|$:

Texture	$\|H - H_{\text{ref}}\|$	PH error	χ	$\|E\|_1$	$\|E\|_3$
all $f = 0$ (FM $\hat{z}$)	3.3e-12	0	5	1.31e-14	0.255
all $f = 1$ (AFM $\pm\hat{y}$)	1.9e-12	0	6	1.38	1.378
$f = 0$ on middle half, 1 outside	1.4e-12	0	8	3.32e-08	0.259

Rotation sweep (§5). Every spin is rotated by the same $f = 0, 0.1, \ldots, 1$ and the dense spectrum of the MPO gives $|E|_1$ (the Majorana pair when it exists) and $|E|_3$ (the bulk gap edge). The zero mode survives up to $f \approx 0.3$ ($|E|_1 = 2.2 \times 10^{-4}$), the gap nearly closes at $f \approx 0.35$, and the chain is trivially gapped beyond ($|E|_1 = 1.378$ at $f = 1$). The sweep is saved as `sweep_f`, `sweep_E13`; Python §2 reproduces it.

5 LDOS and the spin-ramp definitions (TN §6)

`ldos(L, f; η=0.03, ω=range(-3, 3; length=601), model...)`

Exact LDOS from the eigenpairs (E_k, ψ_k) of the densified MPO, $\psi_k = (u_k, v_k)$:

$$A_p(n,\omega) = \sum_k \sum_\sigma |u_k(n,\sigma)|^2\, \delta_\eta(\omega - E_k), \qquad A_h(n,\omega) = \sum_k \sum_\sigma |v_k(n,\sigma)|^2\, \delta_\eta(\omega - E_k),$$

with Lorentzian $\delta_\eta(x) = (\eta/\pi)/(x^2 + \eta^2)$, summed over spin.

```
E, U = eigen(Hermitian(dense(HB, sites)))
wp = abs2.(U[1:N, :])      .+ abs2.(U[N+1:2N, :])    # N × 4N, particle ↑+↓
wh = abs2.(U[2N+1:3N, :])  .+ abs2.(U[3N+1:4N, :])   # N × 4N, hole ↑+↓
lor = (η/π) ./ ((ω .- E') .^ 2 .+ η^2)               # Nω × 4N
return lor * wp', lor * wh'                          # Nω × N each
```

Returns `(A_p, A_h)`, each `length(ω) × N`. Extra keywords go to `msh_chain` (e.g. `ldos(6, n -> 0.0; J=5.0)`). Particle–hole symmetry gives $A_h(n,\omega) = A_p(n,-\omega)$; the total LDOS is $A_p + A_h$.

Asymmetry of A_p. The particle LDOS is concentrated at $\omega > 0$. With $J = 5.8$ the lower exchange-split band spans $\xi \in [-0.8, 3.2]$, so $\mu = -7$ sits only 0.8 above its bottom. Almost all normal-state states are empty, and empty states put their particle weight at positive energy.

5.1 Spin ramp: `smoothstep`, `SpinRamp` and its methods

These are defined in §6 and reused in §10. They implement the CZ project's two-sided ramp:

$$f_n(t) = \mathrm{smoothstep}\Big(\frac{t - t_0}{T_{\mathrm{rot}}} - \mathrm{step}_T\, k_n\Big), \qquad k_n = \min(n, N-1-n),$$

$$\mathrm{smoothstep}(\tau) = \begin{cases} 0 & \tau \le 0 \\ \sin^2(\pi\tau/2) & 0 < \tau < 1 \\ 1 & \tau \ge 1 \end{cases}$$

for the $(N - \ell_{\mathrm{final}})/2$ outermost sites on each side; the central ℓ_{final} sites never rotate. Each site turns over a time T_{rot}, consecutive sites start $\mathrm{step}_T\, T_{\mathrm{rot}}$ apart, so the walls move at $1/(\mathrm{step}_T\, T_{\mathrm{rot}})$ sites per unit time and are about $1/\mathrm{step}_T$ sites wide.

```
Base.@kwdef struct SpinRamp
    N::Int
    ℓ_final::Int = 4
    rot_T::Float64 = 100.0      # CZ rot_Tx
    step_T::Float64 = 0.1       # CZ step_Tx
    t0::Float64 = 0.0
end
```

Function	Definition
`smoothstep(τ)`	$\sin^2(\pi\tau/2)$ clamped to 0 / 1 outside $[0,1]$ (CZ project).
`n_rot(p)`	`(N - ℓ_final) ÷ 2`, sites rotated per side.
`rot_order(p, n)`	$k = \min(n, N-1-n)$ if $k <$ `n_rot`, else `nothing` (site never rotates).
`f_nt(p, n, t)`	$f_n(t)$ above; 0 for non-rotating sites.
`duration(p)`	$T = ((n_{\mathrm{rot}} - 1)\,\mathrm{step}_T + 1)\,T_{\mathrm{rot}}$, the time the last site finishes.
`wall_speed(p)`	$1/(\mathrm{step}_T\, T_{\mathrm{rot}})$ sites per unit time.
`wall_L(p, t)`	Left wall centre, the site whose rotation is halfway (smoothstep $= \frac{1}{2}$ at $\tau = \frac{1}{2}$): $((t - t_0)/T_{\mathrm{rot}} - \frac{1}{2})/\mathrm{step}_T$, clamped to $[-0.5,\, n_{\mathrm{rot}} - 0.5]$. The right wall is $N - 1 -$`wall_L`.

For $N = 64$: with $\ell_{\mathrm{final}} = 4$, $n_{\mathrm{rot}} = 30$ and $T = 390$; with $\ell_{\mathrm{final}} = 0$, $n_{\mathrm{rot}} = 32$ and $T = 410$. The wall speed is 0.10 sites per unit time.

`ldos_ramp`, `ldos_snapshots`. `ldos_ramp = SpinRamp(N=64, ℓ_final=0)` rotates every site, so the chain goes from fully topological ($t = 0$) to fully trivial ($t = T = 410$). `ldos_snapshots` is a list of four (label, f) pairs at $t = 0$, $T/3$, $2T/3$, T; the inner helper `walls(t)` formats the wall positions. The printed labels are: $t = 0$ fully topological; $t = 137$, walls at $n \approx 9, 54$; $t = 273$, walls at $n \approx 22, 41$; $t = 410$ fully trivial. Used in §6, §7a and the save cell.

LDOS plot cell. For each snapshot the cell calls `ldos`, prints $\max|A_h(\omega) - A_p(-\omega)|$ (6.4e-13, 2.2e-12, 2.0e-12, 1.8e-12), and plots three rows: particle LDOS, particle + hole LDOS, and the $\omega = 0$ cut. The local helper `cmax(A)` sets the colour scale to the 99.5th percentile. The figure is saved as `msh_chain_ldos.png` (Fig. 1).

6 LDOS with TensorBinding's KPM routine (TN §7)

Dense diagonalization stops being feasible beyond $N \approx 128$. TensorBinding's `get_ldos_spatial` uses the kernel polynomial method: it expands $\delta(\omega - H)$ in Chebyshev polynomials $T_n(\tilde{H})$ of

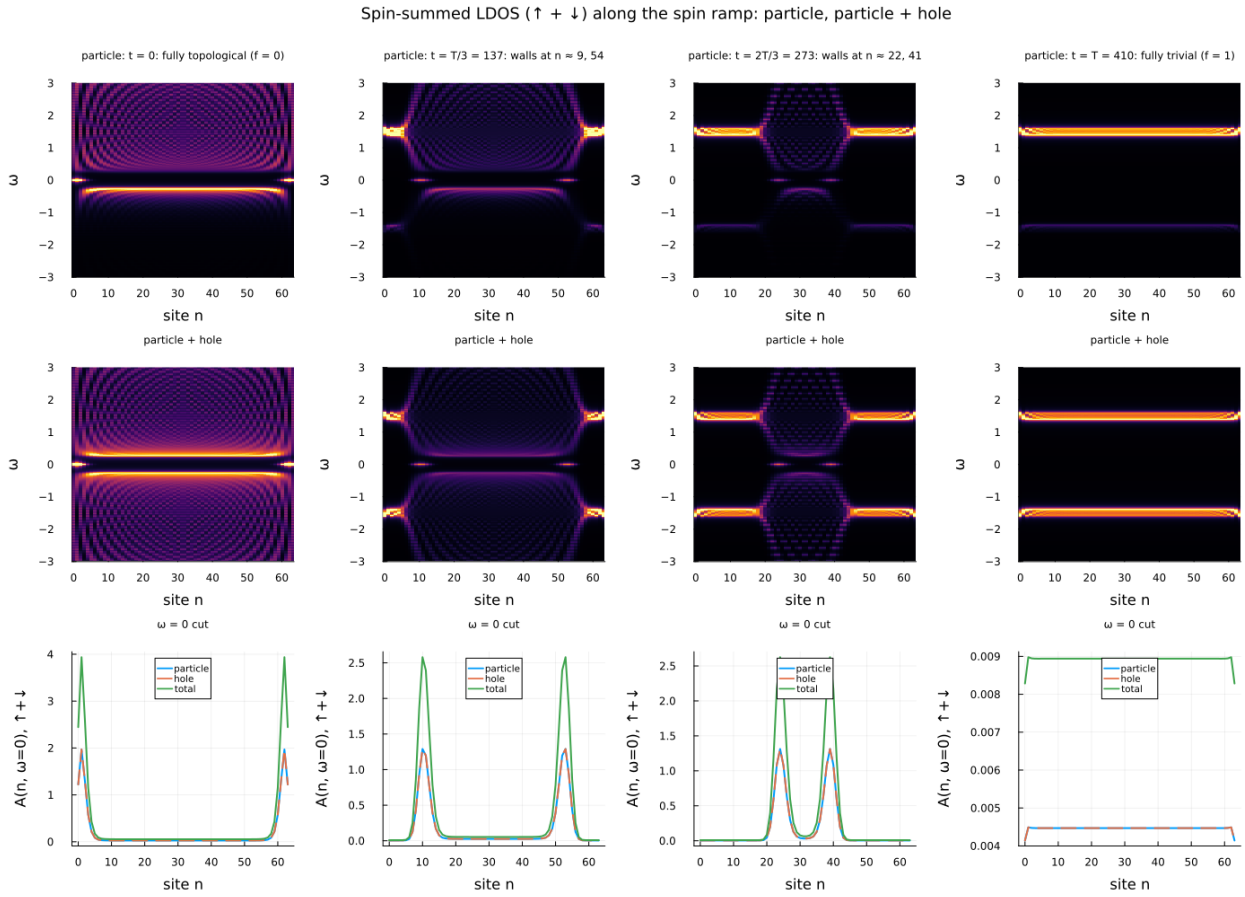


Figure 1: TN §6: spin-summed Lorentzian LDOS ($\eta = 0.03$) at four snapshots of the spin ramp with $\ell_{\text{final}} = 0$. Row 1: particle; row 2: particle + hole; row 3: $\omega = 0$ cut. At $t = 0$ zero-energy peaks sit at both chain ends and the bulk gap is ≈ 0.26; at $T/3$ and $2T/3$ the Majoranas sit at the moving domain walls; at $t = T$ the chain is fully gapped with the band edge near 1.4.

$\tilde{H} = (H - \text{center})/\text{scale}$, evaluated on the MPO, damped with a Jackson kernel. No dense matrix is built.

`msh_tb(L, f; scale=nothing, t, μ, α, Δ, J, model...)`

```
"""
    msh_tb(L, f; scale=nothing, model...) -> TBHamiltonian

Wrap the `msh_chain` BdG MPO as a TBHamiltonian for TensorBinding's KPM routines.
`scale` is the KPM half-bandwidth; by default a safe analytic bound on ‖H_BdG‖.
"""
```

`get_ldos_spatial` takes a `TBHamiltonian`, not a bare MPO. `msh_tb` builds the chain with `msh_chain` and wraps it with `spin_s`, `nambu_s`, `aux_side = :pre`, `center = 0` and `geometry = i -> Float64[i]`. The package's sector projections expect the site order `[nambu; spin; pos...]`, which `msh_chain` produces. The default scale is 1.05× the bound

$$\|H_{\text{BdG}}\| \le |\mu| + |J| + 2\sqrt{t^2 + \alpha^2} + |\Delta|,$$

so the package does not run its DMRG-based estimate. With the defaults, scale = 18.785 (printed by the Python notebook).

Using `get_ldos_spatial`. `nambu_proj=true, proj_nambu=1` selects the particle sector (`2` the hole sector); `spin_proj=true` sums over spin (`proj_s=1` or `2` for one spin). The raw output is per rescaled energy and carries an extra $1/\pi$: multiplying by `π / H.scale` gives the physical LDOS, which integrates to 2 states per site ($\uparrow + \downarrow$) per Nambu sector, the normalization of §6.

Resolution and cost. The Jackson kernel broadens every peak by about $\pi\,\text{scale}/N_{\text{cheb}}$. The spectrum spans about ± 16 but the gap is only 0.26, so $N_{\text{cheb}} \approx 1000$ is needed.

Table 4: KPM cost on 64 sites (from the notebook markdown).

`Ncheb`	resolution	mode	time (64 sites)
128	≈ 0.46	`:mpo` (whole chain in one pass)	≈ 25 s
1024	≈ 0.058	`:mpo`	$\approx$ 5–11 min (texture-dependent)
1024	≈ 0.058	`:mps` (one recursion per site and spin)	≈ 6 s per site

At $N = 64$ this is slower than dense diagonalization; the advantage is scaling. `mode=:mps` with `x_groups` computes only selected sites (chain ends, domain walls).

`kpm_exact(H::TB.TBHamiltonian, Ncheb, ω; kernel=:jackson, proj_nambu=1)`

Pushes the exact eigenpairs through the same Chebyshev formula as `get_ldos_spatial` (same kernel, N_{cheb} and normalization), so any difference from `get_ldos_spatial` is pure tensor-network arithmetic error, not broadening. With $\tilde{E}_k = (E_k - \text{center})/\text{scale}$, $\tilde{\omega} = (\omega - \text{center})/\text{scale}$ and $w_k(x)$ the spin-summed weight of eigenvector k in the chosen Nambu sector:

$$\mu_n(x) = \sum_k w_k(x)\, T_{n-1}(\tilde{E}_k), \qquad A(x,\omega) = \frac{\sum_n W_{n,\tilde{\omega}}\, \mu_n(x)}{\pi^2 N_{\text{cheb}} \sqrt{1-\tilde{\omega}^2}},$$

with W = `TB._kpm_weight_matrix(Ncheb, ω̃, kernel)` and $A = 0$ for $|\tilde{\omega}| \ge 1$. T_n is evaluated as $\cos(n \arccos \tilde{E})$ with $\tilde{E}$ clamped to $[-1, 1]$. Returns the raw (package-unit) LDOS, $N_\omega \times N$; multiply by π/scale for physical units. The Python port of W (Section 15) shows its content: $W_{n,\tilde{\omega}} = c_n\, g_n\, T_n(\tilde{\omega})$, $c_0 = 1$, $c_{n>0} = 2$, g_n the unnormalized Jackson factors.

§7a cell. With `RUN_FULL_KPM = false` it loads `msh_chain_ldos_kpm_ramp.jls`; otherwise, for each of the four `ldos_snapshots` on $L = 6$, it computes on 241 points in $[-3, 3]$:

```
A_tb = TB.get_ldos_spatial(H, Ncheb, ω; nambu_proj=true, spin_proj=true,
                           maxdim=512, cutoff=1e-12) .* (π / H.scale)
A_ex = (kpm_exact(H, Ncheb, ω; proj_nambu=1) .+ kpm_exact(H, Ncheb, ω; proj_nambu=2)) .* (π /
↪  H.scale)
```

with $N_{\text{cheb}} = 1024$; `proj_nambu` is omitted in `get_ldos_spatial`, so both Nambu sectors are summed. Printed $\max|A_{\text{TB}} - A_{\text{exact}}|$: 5.3e-06, 3.8e-06, 3.6e-06, 3.4e-05. The Jackson broadening is roughly Gaussian with width ≈ 0.058, while §6 uses a Lorentzian with $\eta = 0.03$, so peak positions agree with §6 but peak heights do not (Fig. 2).

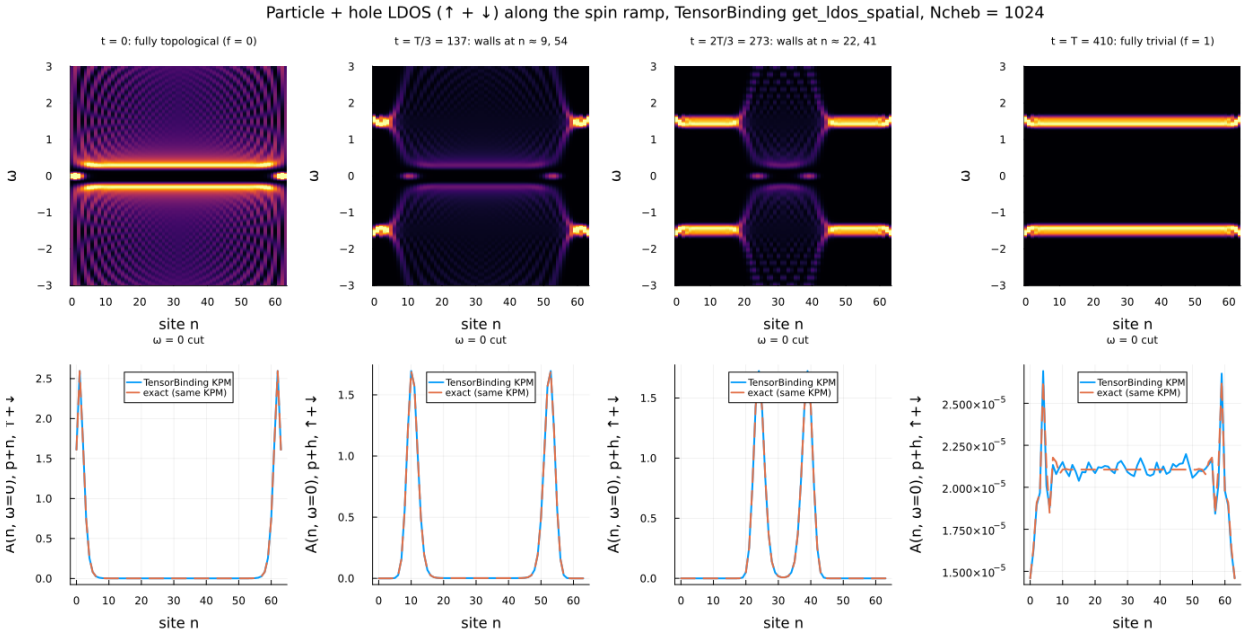


Figure 2: TN §7a: particle + hole LDOS from `get_ldos_spatial`, $N_{\text{cheb}} = 1024$ (row 1), and the $\omega = 0$ cut against `kpm_exact` (row 2). In the trivial snapshot the $\omega = 0$ value is $\sim 2 \times 10^{-5}$ and the MPS arithmetic error is visible.

7 Majorana zero modes and the fermion mode (TN §8)

Following `kitaev_tensor_network_V2.ipynb`, four steps turn the Majorana pair into one ordinary fermion using only MPS/MPO operations:

1. **Zero-mode subspace.** Penalized DMRG on the folded operator $K = H^2 + \eta^2$ finds the two states closest to $E = 0$. A third penalized state is a gap check and should land at the bulk gap.
2. **Localize.** Diagonalize the position operator $\hat{X}$ inside the two-state subspace. Its eigenvectors are the left and right Majoranas γ_1, γ_2.
3. **Self-conjugacy.** With $C = \tau_x K$ (K = complex conjugation), each $\hat{X}$ eigenvector satisfies $C\gamma = e^{i\theta}\gamma$; multiplying by $e^{i\theta/2}$ makes $C\gamma = \gamma$.
4. **Pair.** $\psi_f = (\gamma_1 + i\gamma_2)/\sqrt{2}$, choosing the member with $E_f \geq 0$; its partner is $\bar{\psi}_f = C\psi_f$.

Differences from the spinless Kitaev notebook: the position qubits are `sites[3:end]`, and C acts as τ_x on the Nambu site only (identity on spin).

Numerical settings that matter (each found by comparison with exact diagonalization):

- K's spectrum spans about 10^{-4} to 230 ($\|H\| \approx 15$ against a gap of 0.26). ITensor's default local Krylov dimension (3) stalls at $\|H\psi\| \approx 0.2$; `eigsolve_krylovdim = 20` converges in about a second.
- `TB.build_K` needs `cutoff_K` $\leq 10^{-14}$. The package default (10^{-8}) truncates part of the η^2 term.
- DMRG on K leaves a bulk admixture of amplitude $\sim 10^{-7}$–10^{-6}. This does not matter for energies or profiles but does matter for ΔQ when the pair splitting is tiny; `refine_subspace` removes it (§9).

7.1 Small helpers

Function	What it does
`quietly(f)`	Runs `f` with a logger that shows only errors. KrylovKit warns "ignoring imaginary component" at every DMRG/linsolve/TDVP step on these MPOs; the imaginary parts are round-off ($\sim 10^{-9}$ of the weight).
`norm_Hψ(H, ψ)`	$\|H\psi\|$ computed exactly: `apply(H, ψ; alg="naive", truncate=false)`, then the norm of the orthogonality centre. Tiny values are a cancellation of $O(1)$ terms, so no truncation is allowed.
`dense_vec(ψ, sites)`	Contracts an MPS into a dense vector ordered like `dense()` (nambu ⊗ spin ⊗ position, big-endian). Small L only; used by the optional exact checks.
`apply_site(op, ψ, j)`	Copy of `ψ` with the single-site operator `op` applied to site `j`.

`uv_profile(ψ, sites)`

Returns the spin-summed particle weight $u(n) = \sum_\sigma |u_\sigma(n)|^2$ and hole weight $v(n) = \sum_\sigma |v_\sigma(n)|^2$ for each site of an MPS on `[nambu; spin; pos₁ … pos_L]`, without forming the $4 \cdot 2^L$ vector. For site n it fixes each position qubit to the bits of n (`onehot`, `pos₁` most significant) and contracts with the Nambu and spin tensors:

```
head = ψ[1] * ψ[2]
for n in 0:2^L-1
    T = head
    for k in 1:L
        T *= ψ[k+2] * onehot(pos[k] => ((n >> (L - k)) & 1) + 1)
    end
    A = Array(T, ns, ss)
    u[n+1], v[n+1] = sum(abs2, A[1, :]), sum(abs2, A[2, :])
end
```

Cost $\sim 2^L \cdot L \cdot \chi^2$, so it works at any L. Used for the Majorana and ψ_f profiles (§8a, save cell) and for $|\psi_f(x,t)|^2$ and $\delta n(x,t)$ in §10a.

7.2 The pipeline

`zero_mode_subspace(HB, sites; nzero=2, η=1e-2, ...)`

```
function zero_mode_subspace(HB, sites; nzero=2, η=1e-2, seed=1234,
                            nsweeps=20, maxdim=[10, 20, 50, 100, 200], cutoff=1e-14,
                            ↪ cutoff_K=1e-14,
                            noise=[1e-6, 1e-7, 1e-8, 0.0], weight=10.0,
                            eigsolve_krylovdim=20, eigsolve_maxiter=3)
```

Builds $K = H^2 + \eta^2$ with `TB.build_K(HB, sites, 0.0, η; cutoff_K)` and runs `nzero + 1` DMRG calculations from random MPS (bond dimension 10, fixed `seed`). The first is plain DMRG on K;

each later one is penalized DMRG `dmrg(K, ψs, ψ0; weight)`, which adds a penalty `weight` for overlap with the states already found. The last state is removed as the gap check. Returns `(ψs, E_sub, E_gap)`: the `nzero` normalized states, their $\|H\psi\|$, and $\|H\psi\|$ of the gap-check state (should sit at the bulk gap). The settings `eigsolve_krylovdim = 20` and `cutoff_K = 1e-14` are the ones discussed above.

`refine_subspace(HB, ψs; σ=1e-3, nsweeps=6, cutoff=1e-20, maxdim=200)`

One inverse-iteration step per state,

$$\psi \leftarrow \frac{(H-\sigma)^{-1}\psi}{\|(H-\sigma)^{-1}\psi\|},$$

solved with `ITensorMPS.linsolve(HB, ψ, ψ, -σ, 1.0; ...)`. It suppresses the bulk admixture left by DMRG on K by $\sim \mathrm{gap}/\sigma$ while keeping the pair ($|E| \ll \sigma$) in the subspace. The docstring states it should be used only when the pair splitting is far below σ, and that a second step does not help.

`fermion_from_subspace(HB, sites, ψs; cutoff=1e-14)`

Steps 2–4. Returns a named tuple `(centers, γs, Hγ, C)` and, for two states, also `ψf, ψfbar, E_f`.

Localization. The position operator is $\hat{X} = 1 + \sum_{k=1}^{L} 2^{L-k} n_k$, with $n_k = |1\rangle\langle 1|$ on position qubit k (site $k+2$); it is diagonal with eigenvalue $x+1$ on site x. With the overlap matrix $S_{ab} = \langle\psi_a|\psi_b\rangle$ and $X_{ab} = \langle\psi_a|\hat{X}|\psi_b\rangle$ the generalized eigenproblem $Xc = \lambda Sc$ is solved; the eigenvalues are the Majorana centres $\langle x\rangle$ (1-indexed; callers subtract 1), and each eigenvector c gives $\gamma = \sum_j c_j \psi_j$ (MPS sum with `cutoff`), normalized. `eigen` returns the eigenvalues in ascending order, so the γ's are sorted by position.

Self-conjugacy. $\hat{X}$ is real and acts trivially on Nambu and spin, so it commutes with C, and each eigenvector obeys $C\gamma = e^{i\theta}\gamma$. The code implements `C(ψ)` as `dag(ψ)` (complex conjugation) followed by τ_x on site 1, and `fixphase(γ) = exp(im*angle(⟨γ|Cγ⟩)/2)*γ`.

Couplings. $H_{ij} = \langle\gamma_i|H|\gamma_j\rangle = iA_{ij}$ with A real antisymmetric; the check is $\max|\mathrm{Re}\, H_{ij}| \approx 0$.

Pairing. $\psi_f = (\gamma_1 + i\gamma_2)/\|\cdot\|$, $E_f = \mathrm{Re}\langle\psi_f|H|\psi_f\rangle$; if $E_f < 0$, $\psi_f \to C\psi_f$ and $E_f \to -E_f$. `ψfbar = C(ψf)`.

`majorana_fermion(HB, sites; refine_σ=nothing, cutoff=1e-14, kw...)`

Full pipeline: `zero_mode_subspace` → optional `refine_subspace` with σ = `refine_σ` (cutoff $\min(\text{cutoff}, 10^{-20})$) → `fermion_from_subspace`. Returns `ψs, E_sub, E_gap, centers, γs, Hγ, C` and, for two zero modes, `ψf, ψfbar, E_f`. Used in §8a, the save cell and `run_spin_ramp`. Python equivalent: `majorana_fermion(H)` (Section 16).

7.3 §8a example: chain ends and domain walls

Two textures on 64 sites: all spins along $\hat{z}$, and a topological middle half ($16 \le n < 48$) inside a trivial background. Everything except the optional dense check (`check_exact = L ≤ 8`) uses only MPS/MPO operations, so the cell runs unchanged for larger L. The dense check computes the weight of $\gamma_1, \gamma_2, \psi_f$ inside the exact zero-mode pair (via `dense_vec`).

Table 5: Printed results of TN §8a.

	$f = 0$ (ends)	domain (walls)
zero-mode $\|H\psi\|$ (both states)	5.62e-8	6.35e-7
gap-check $\|H\psi\|$	0.255	0.259
centres $\langle x\rangle$ (0-indexed)	1.41, 61.59	16.55, 46.45
$\|C\gamma_i - \gamma_i\|$	0, 5.7e-15	5.1e-10, 5.2e-10
$\max\|\langle\gamma_i\|\gamma_j\rangle - \delta_{ij}\|$	1.1e-15	1.6e-15
$\max\|\mathrm{Re}\, H_{ij}\|$	2.0e-24	5.3e-17
normalization $\sum\|\gamma_1\|^2, \sum\|\gamma_2\|^2, \sum(\|u\|^2+\|v\|^2)$	1, 1, 1 (12 digits)	1, 1, 1
$E_f = \langle\psi_f\|H\|\psi_f\rangle$	1.375e-14	3.319e-08
$\|\langle\psi_f\|\bar{\psi}_f\rangle\|$	2.4e-14	3.6e-12
exact E_f	1.668e-14	3.319e-08
weight in exact pair $(\gamma_1, \gamma_2, \psi_f)$	1.000000000000	1.000000000000

The $\|H\psi\|$ of the DMRG states (5.62e-8, 6.35e-7) is much larger than E_f; the notebook attributes this to the bulk admixture, which inflates $\|H\psi\|$ while $\langle\psi_f|H|\psi_f\rangle$ is only second order in it (§9).

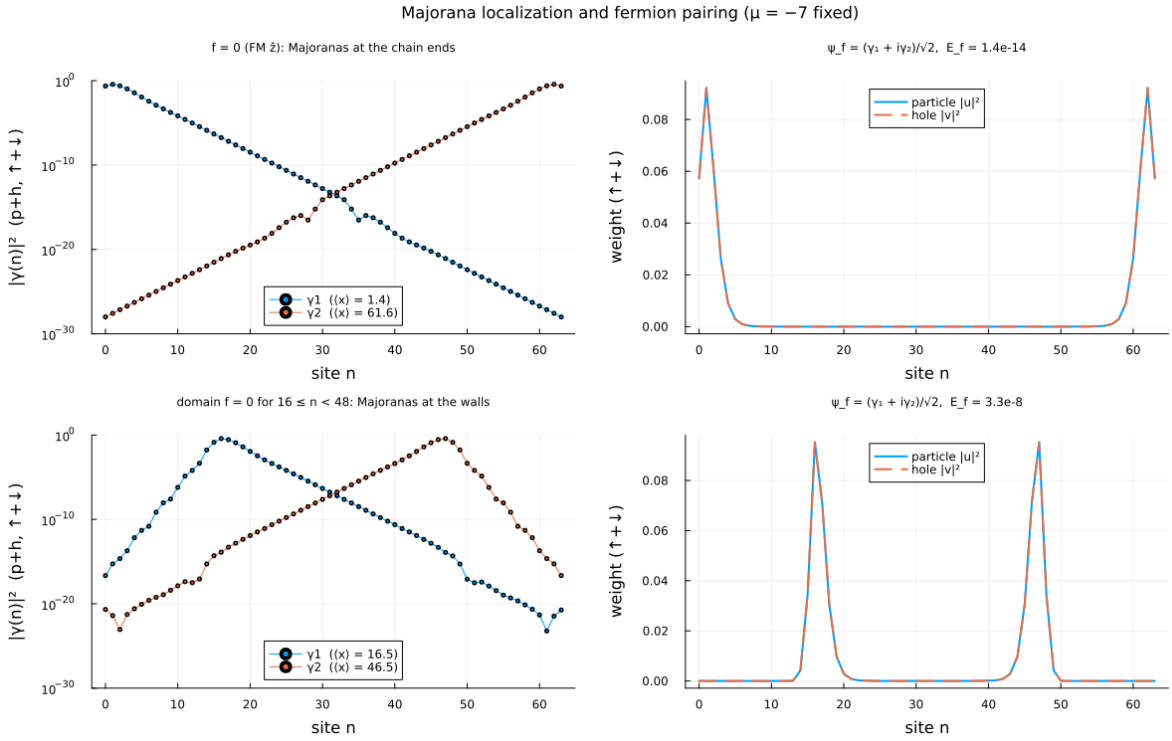


Figure 3: TN §8a: $|\gamma_i(n)|^2$ (particle + hole, $\uparrow + \downarrow$) on a log scale (left) and the particle and hole weights of ψ_f (right), for $f = 0$ (top) and the domain texture (bottom).

8 Charge difference between the logical states (TN §9)

The logical states $|0\rangle$ (mode empty) and $|1\rangle$ (mode filled) are BCS states that share the filled bulk and differ only in which member of the zero-mode pair is occupied. Their BdG correlation matrices differ by exactly $|\psi_f\rangle\langle\psi_f| - |\bar{\psi}_f\rangle\langle\bar{\psi}_f|$, so no bulk projector is needed. With $\psi_f = (u, v)$:

$$\delta n(x) = n_1(x) - n_0(x) = \sum_\sigma |u_\sigma(x)|^2 - |v_\sigma(x)|^2, \qquad \Delta Q = \sum_x \delta n(x) = \langle\psi_f|\tau_z|\psi_f\rangle.$$

δn counts electrons (the charge density is $-e\,\delta n$). $|1\rangle$ is the $+E$ member, which fixes the sign of ΔQ. ΔQ is not quantized; it comes from the overlap of the two Majoranas. Since H contains $-\mu\hat{N}$,

Feynman–Hellmann gives the independent check $\Delta Q = -\partial E_f / \partial \mu$.

Geometry. A centred segment of ℓ sites is topological ($f = 0$), the rest trivial ($f = 1$), at fixed $\mu = -7$. The Majoranas sit at the two domain walls, so shrinking ℓ pushes them together.

Function	What it does
`τz_on(ψ)`	Copy of `ψ` with $\tau_z = \mathrm{diag}(1, -1)$ applied on the Nambu site (identity on spin).
`ΔQ_mode(ψ)`	$\mathrm{Re}\langle\psi|\tau_z|\psi\rangle$.
`component_mps(sites, nb, σ, x)`	Product MPS $\|nb, \sigma, x\rangle$ (one-hot on every site; x 0-indexed, `pos`$_1$ most significant bit).
`δn_mode(ψ, sites)`	$\delta n(x) = \sum_\sigma \|\langle p, \sigma, x|\psi\rangle\|^2 - \|\langle h, \sigma, x|\psi\rangle\|^2$ from overlaps with `component_mps`; no dense vector, so it works at any size.
`segment(N, ℓ)`	0-indexed range `((N-ℓ)÷2):((N-ℓ)÷2+ℓ-1)` of the centred segment.
`f_segment(N, ℓ)`	`n -> n in segment(N, ℓ) ? 0.0 : 1.0`.

`majorana_pair_charge(L, ℓ; refine_σ=1e-3, refine_below=refine_σ/500, gap_factor=2.0, cutoff=1e-20, with_δn=false, model_kw...)`

> `Zero-mode pair (penalized DMRG on K) → gap check → [one inverse-iteration step if the pair splitting ⟨ψ_f|H|ψ_f⟩ < `refine_below`] → X̂ localization → Cγ = γ → ψ_f (+E member) → ΔQ = ⟨ψ_f|τ_z|ψ_f⟩. `ΔQ_raw` is the value without refinement. If the third penalized state is not well above the pair, it warns and returns NaN.`

Steps. Build `msh_chain(L, f_segment(N, ℓ))`; `zero_mode_subspace` with `cutoff = cutoff_K = 1e-20`; gap check $E_{\mathrm{gap}} >$ `gap_factor` $\cdot \max E_{\mathrm{sub}}$ (else warning and `NaN`); `fermion_from_subspace` → `r0`; if $|E_f| <$ `refine_below`, redo `fermion_from_subspace` on `refine_subspace(ψs)`. Returns `(ΔQ, ΔQ_raw, refined, E, E_next, δn, centers)`; `δn` only if `with_δn`.

Why the refinement rule. ΔQ is first order in any bulk admixture, because τ_z couples the zero mode to bulk states, while E_f is only second order. DMRG alone gives errors of about 10^{-8}, growing to 10^{-6} once ΔQ itself is tiny ($\ell \gtrsim 30$). One inverse-iteration step ($\sigma = 10^{-3}$) brings the error to 10^{-10}–10^{-13}, but only when the pair splitting is far below σ. For $E \gtrsim \sigma/500$ the `linsolve` on $H - \sigma$ is poorly converged and adds error (up to 10^{-4}), because $(H - \sigma)^{-1}$ weights the $\pm E$ members unequally, while DMRG alone is already accurate. The splitting is judged by $\langle\psi_f|H|\psi_f\rangle$ (second order in the admixture), not by $\|H\psi\|$ of the DMRG states, which the admixture inflates. With the defaults the threshold is 2×10^{-6}.

`exact_pair_charge(L, ℓ; dμ=1e-4, μ=-7.0, model_kw...)`

Dense reference from `reference`. The $+E$ state is the first positive eigenvalue (`findfirst(>(0), E)`), reshaped as `(x, spin, nambu)`; $\delta n(x)$ and $\Delta Q = \sum_x \delta n$ follow from the formula above. The Feynman–Hellmann value is the central difference

$$\Delta Q_{\mathrm{FH}} = -\frac{E_+(\mu + d\mu) - E_+(\mu - d\mu)}{2\, d\mu}, \qquad d\mu = 10^{-4}.$$

Returns `(ΔQ, E, δn, ΔQ_FH)`.

8.1 §9a: ΔQ versus segment length

$\ell = 2, 4, \ldots, 48$ on 64 sites; cached in `msh_chain_charge.jls` (`RUN_CHARGE`). Each point is compared with `exact_pair_charge`. Points with $\ell \geq 24$ are refined ($E_f < 2 \times 10^{-6}$); unrefined points take about 6 s, refined points 42–73 s.

Table 6: Selected rows of the TN §9a output.

ℓ	ΔQ_{MPS}	$\Delta Q_{\mathrm{exact}}$	$-\partial E/\partial\mu$	error (DMRG only)	refined
2	+7.214509e-01	+7.214509e-01	+7.215e-01	-3.7×10^{-9} (-3.7×10^{-9})	
10	+5.012841e-02	+5.012842e-02	+5.013e-02	-8.4×10^{-9} (-8.4×10^{-9})	
16	+4.043709e-03	+4.043733e-03	+4.044e-03	-2.3×10^{-8} (-2.3×10^{-8})	
24	+1.126176e-04	+1.126177e-04	+1.126e-04	-2.9×10^{-12} (-3.9×10^{-10})	yes
30	+7.112500e-06	+7.128537e-06	+7.129e-06	-1.6×10^{-8} (-1.2×10^{-8})	yes
36	+4.333961e-07	+4.342232e-07	+4.342e-07	-8.3×10^{-10} (-1.2×10^{-7})	yes
48	+1.672126e-09	+1.496951e-09	+1.503e-09	$+1.8\times10^{-10}$ $(+1.5\times10^{-6})$	yes

The refinement lowers the error by one to four orders of magnitude for $\ell \geq 32$. At $\ell = 48$ the remaining error (1.8×10^{-10}) is about 12 % of ΔQ itself (1.5×10^{-9}).

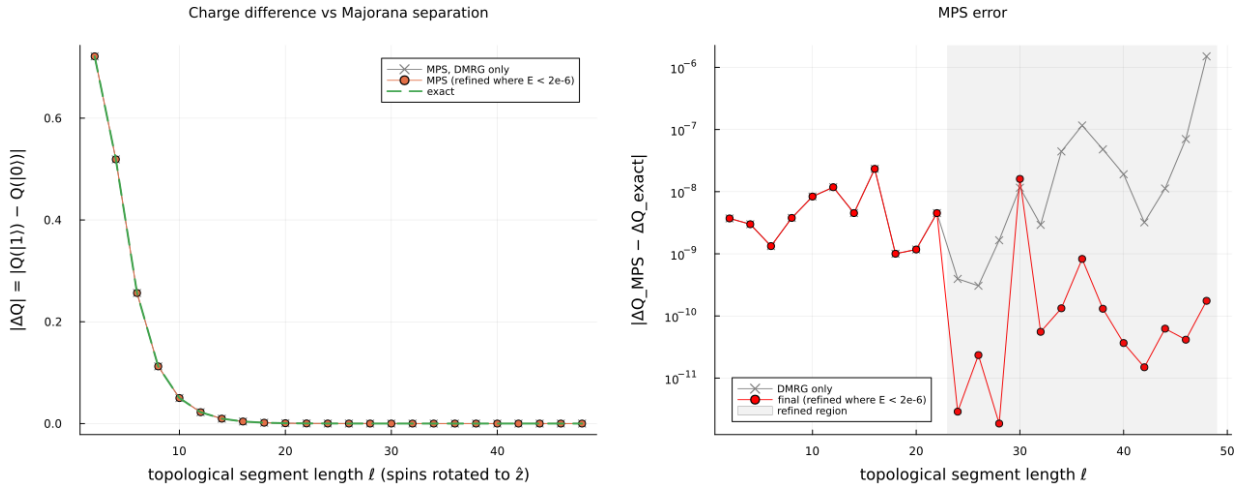


Figure 4: TN §9a. Left: $|\Delta Q|$ vs ℓ from MPS (DMRG only, and refined where $E < 2\times10^{-6}$) and exact diagonalization. Right: $|\Delta Q_{\mathrm{MPS}} - \Delta Q_{\mathrm{exact}}|$; the shaded region marks the refined points.

9 Time-dependent ramp (TN §10)

At $t = 0$ every spin points along $\hat{z}$ and the Majoranas sit at the chain ends. Sites are rotated one after another, from both ends, into the in-plane AFM texture; the two domain walls move inward and carry the Majoranas toward the centre, stopping when ℓ_{final} topological sites are left. This is the Kitaev notebook's "moving the MZMs" calculation with $\mu(x,t)$ replaced by spin rotation. The ramp is `SpinRamp` / `f_nt` (Section 4).

What is evolved. Only the fermion mode. $\Gamma_1(t) - \Gamma_0(t) = |\psi_f(t)\rangle\langle\psi_f(t)| - |C\psi_f(t)\rangle\langle C\psi_f(t)|$ at all times (the shared bulk cancels and $U(t)$ commutes with C), so

$$\delta n(x,t) = \sum_\sigma |u_\sigma(x,t)|^2 - |v_\sigma(x,t)|^2, \qquad \Delta Q(t) = \langle\psi_f(t)|\tau_z|\psi_f(t)\rangle,$$

with $i\partial_t\psi = H(t)\psi$. At $t = T$ the weight of ψ_f outside the instantaneous $\pm E$ doublet is the *leakage* into bulk quasiparticles (the non-adiabatic error). $|1\rangle$ is the state whose mode ends in $+E$; if ψ_f ends in $-E$ the roles swap and ΔQ changes sign.

`RampHamiltonian(L, p::SpinRamp; J=5.8, φ=π/2, model_kw...)`

```
struct RampHamiltonian
    L::Int; p::SpinRamp; J::Float64; φ::Float64; model_kw
    H_static::MPO; sites::Vector{<:Index}
end
```

A callable struct: `R(t; cutoff=1e-14)` returns the BdG MPO $H(t) = H_{\text{static}}$+ exchange(t). H_{static} (hopping, μ, Rashba, pairing) is built once with `msh_chain(L, n -> 0.0; J=0.0)`. At each call the exchange is rebuilt exactly: for each component, `diag_mpo_exact` of $JS_n^k(t)$, entering the particle block as σ_k (`:Pp`) and the hole block as $-\sigma_k^*$ (`:Ph`).

Helpers: `f_at(R, t)` returns `n -> f_nt(R.p, n, t)`; `dense_H(R, t)` is the dense reference `reference(R.L, f_at(R, t); J, φ, model_kw...)`.

Check. For $L = 6$, $\ell_{\text{final}} = 4$, $T = 390$: $\|H_{\text{MPO}} - H_{\text{dense}}\|$ = 1.2e-12, 4.9e-12, 3.4e-12, 1.3e-12 at $t = 0, 130, 260, 390$, with MPO bond dimensions 5, 12, 12, 10. The printed profile at $t = 150$ shows fully rotated sites near the ends, a wall about 10 sites wide, and unrotated sites in the middle.

`final_branches(H, sites; refine_σ=1e-3)`

Instantaneous $\pm E$ doublet of an MPO Hamiltonian: `zero_mode_subspace` (cutoffs 10^{-20}), one `refine_subspace` step, then a 2×2 Ritz problem $H_s c = ESc$ in the refined pair. Returns ψ_+, ψ_- and E_+.

`run_spin_ramp(R; dt=0.1, nsnap=200, maxdim=64, cutoff=1e-20, tol=1e-12, krylovdim=30)`

```
1. ψ_f(0) from the t = 0 Hamiltonian (zero-mode pair + one refinement step, localization,
   Cγ = γ, pairing).
2. TDVP with H frozen at the midpoint of each step.  reverse_step = true is required: the
 ↪  package
   default (false) skips TDVP's backward sub-steps and gives wrong dynamics.
3. At t = T: overlaps with the instantaneous ±E doublet → leakage 1 − p₊ − p₋.
Returns ΔQ(t) and `nsnap + 1` snapshots of ψ_f.
```

Details. $n_{\text{steps}} = \text{round}(T/dt)$ and dt is reset to T/n_{steps}. $\psi_f(0)$ comes from `majorana_fermion(R(0.0), sites; refine_σ=1e-3, cutoff=1e-20, cutoff_K=1e-20)`. The evolution is `TB.evolve_with_tdvp_timedep(t -> R(t), r0.ψf, nsteps, dt; maxdim, cutoff, tol, krylovdim, reverse_step=true)`. At $t = T$, $p_\pm = |\langle\psi_\pm|\psi(T)\rangle|^2$ from `final_branches`; if $p_- > p_+$ the sign $\sigma = -1$ is applied to ΔQ and $p_\pm$ are swapped. Samples are at `unique(round.(Int, range(1, nsteps + 1; length=nsnap + 1)))`. Returns `T, dt, E0, leak, p_wrong, E₊, ΔQ_final, ΔQ_adiabatic` ($= \Delta Q$ of ψ_+), `times, ΔQ_t, snapshots, σ, maxχ`.

`run_spin_ramp_exact(R; dt=0.1, nsnap=200)`

Dense reference with the same midpoint propagator, initial pairing and end-of-ramp analysis:

$$\psi \leftarrow \exp(-i\,dt\,H(t_k - dt/2))\psi, \qquad t_k = k\,dt.$$

$\psi_f(0)$ is built from the two smallest-$|E|$ eigenvectors of `dense_H(R, 0)`: localized by the position operator `Diagonal(repeat(0:N-1, 4))`, phase-fixed with C implemented as `Cd(v) = vcat(conj(v[2N+1:end]), conj(v[1:2N]))` ($\tau_x K$), paired as $(\gamma_1 + i\gamma_2)/\|\cdot\|$, and replaced by $C\psi$ if its energy is negative. $\Delta Q = \langle\psi|\tau_z|\psi\rangle$ with $\tau_z = \text{diag}(\mathbb{1}_{2N}, -\mathbb{1}_{2N})$ is recorded at the same samples. At T the leakage uses the dense eigenvectors at the first positive eigenvalue and the one below. Returns `leak, p_wrong, ΔQ_final, ΔQ_t, ψ_t, σ`.

`adiabatic_reference(R, t) -> (ΔQ_ad, E_gap)`

ΔQ of the instantaneous $+E$ state of $H(t)$ and the bulk gap (third smallest $|E|$), from the dense matrix. The pair is the two levels with the smallest $|E|$. While the Majoranas are far apart, the splitting falls below round-off and the two members are numerically degenerate, so "the $+E$ state" would be an arbitrary mix with an $O(1)$ ΔQ; the true value is then below $\sim 2 \times 10^{-9}$, so ΔQ_{ad} is

set to 0 when $E_+ < 10^{-11}$. The bound follows from Feynman–Hellmann (as the docstring now explains): with $E \propto e^{-\kappa\ell}$, $\Delta Q = -\partial E/\partial\mu \approx (\partial\kappa/\partial\mu)\,\ell E$, and the §9a data give $\Delta Q/(\ell E) \to 2.6$ (2.63 at $\ell = 48$), so $E < 10^{-11}$ and $\ell \le 64$ mean $\Delta Q \lesssim 2 \times 10^{-9}$. The notebook recommends this function over `findfirst(>(0), E)` for that reason.

9.1 §10a: the ramp on 64 sites

`ramp_params = (L = 6, rot_T = 100.0, step_T = 0.1, ℓ_final = 4)`, $T = 390$, cached in `msh_chain_ramp.jls` (`RUN_RAMP`). The cell runs `run_spin_ramp` and `run_spin_ramp_exact`, computes $W = |\psi_f(x,t)|^2$ and δn with `uv_profile` on the MPS snapshots (times σ), the dense δn from `ψ_t`, and `adiabatic_reference` at every sample. Only derived quantities are saved, not MPS snapshots.

Table 7: Printed results of TN §10a.

	MPS / TDVP	exact
$E(0)$	1.3e-14	—
leakage $1 - p_+ - p_-$	2.143e-04	2.143e-04
wrong branch p_-	3.7e-18	4.1e-27
$\Delta Q(T)$	+0.516615	+0.516615
adiabatic $\Delta Q(T)$	+0.519062	—
max bond dimension	16	—
run time	177 s	49 s
$\max_t \lvert\Delta Q_{\mathrm{MPS}} - \Delta Q_{\mathrm{exact}}\rvert$	4.1e-07	
$\max_{x,t} \lvert\delta n_{\mathrm{MPS}} - \delta n_{\mathrm{exact}}\rvert$	2.6e-07	

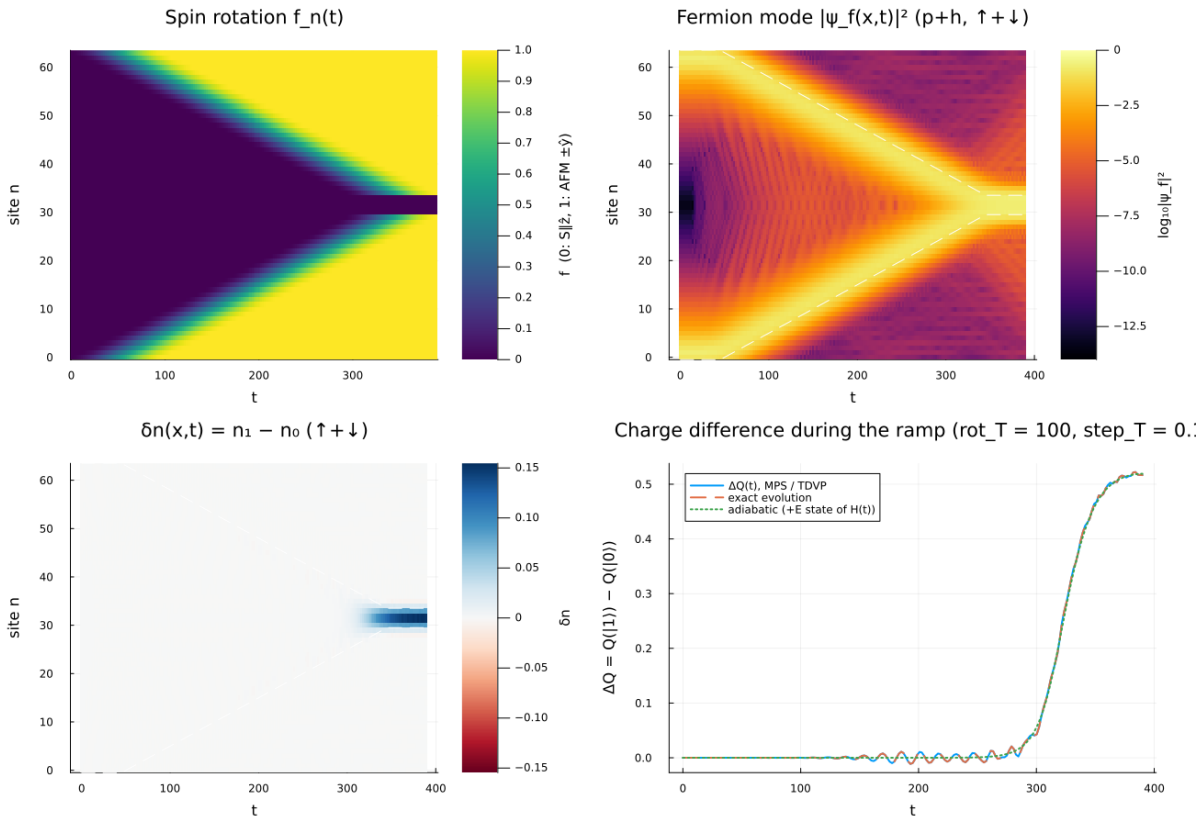


Figure 5: TN §10a. Top left: rotation profile $f_n(t)$. Top right: $|\psi_f(x,t)|^2$ (log scale); it rides the two walls (dashed: wall centres) into the middle. Bottom left: $\delta n(x,t) = n_1 - n_0$, exponentially small until the Majoranas approach each other. Bottom right: $\Delta Q(t)$ from TDVP, exact evolution and the instantaneous adiabatic value. The notebook describes the wiggles of about ± 0.01 mid-ramp as physical (interference with the small bulk component the ramp excites).

10 How to adapt the notebook (TN §11)

- **Spin texture:** pass any `f(n)`, e.g. `f_wall(n) = n < 20 ? 1.0 : 0.0` (single wall) or `f_ramp(n) = clamp((n - 16) / 32, 0, 1)`. For time-dependent protocols use `RampHamiltonian`, which rebuilds only the exchange.
- **Parameters:** keyword arguments, e.g. `msh_chain(6, f; μ=-6.5, J=5.0, Δ=2.0)`; the same keywords go to `ldos`.
- **Bigger chains:** `msh_chain` works at large L (small bond dimension; $O(2^L)$ memory for the diagonal MPOs). `dense`, `reference`, `ldos` need $L \leq 7$. Use `majorana_fermion/majorana_pair_charge`, `run_spin_ramp` and `msh_tb` + `get_ldos_spatial` directly; `uv_profile` and `δn_mode` work at any size; the `exact_*` functions and `adiabatic_reference` are dense checks only.
- **Pitfalls:** do not add `add_superconductivity!`, `add_zeeman!` or `add_soc!` terms (τ_z hole block, periodic Rashba bond); `:Sx/:Sy/:Sz` are $\sigma/2$; build sharp profiles with `diag_mpo_exact`, not QTCI, for $L \geq 8$; pass `reverse_step = true` to TDVP; refine only when the pair splitting is far below σ; use `adiabatic_reference` for the $+E$ state; regenerate the `.jls` caches after changing definitions.
- **Ramp:** `SpinRamp(N=N, rot_T=..., step_T=..., ℓ_final=...)`; edit `f_nt` for a different rotation order (one end only, a single wall); `ramp_params` in §10a sets the saved run.

11 Saving results (TN, last section)

The final cell (`using NPZ`) writes 43 arrays to `msh_chain_tn_results.npz`. The sweep and Lorentzian LDOS are recomputed; KPM, charge and ramp arrays come from the cached `kpm`, `charge` and `ramp` results, and the Majorana arrays from a fresh `majorana_fermion` run. Centres are stored 0-indexed (`r.centers .- 1`).

Keys	Content (section)
`sweep_f`, `sweep_E13`	$\|E\|_1, \|E\|_3$ of the densified MPO vs f (§5)
`ldos_omega`, `ldos_p_j`, `ldos_h_j`	Lorentzian particle/hole LDOS, snapshots $j = 0 \dots 3$ (§6)
`kpm1024_omega`, `kpm1024_tb_j`, `kpm1024_exact_j`	p + h KPM LDOS: `get_ldos_spatial` and `kpm_exact` (§7a)
`mzm_centers_j`, `mzm_gamma_weight_j`, `mzm_E_f_j`	DMRG Majoranas, $j = 0$: $f = 0$, $j = 1$: domain (§8a)
`charge_ells`, `charge_dQ_mps`, `charge_dQ_mps_raw`, `charge_dQ_exact`, `charge_dQ_FH`, `charge_E_exact`, `charge_dn_exact`	$\Delta Q(\ell)$: MPS refined / DMRG only, exact, $-\partial E_f/\partial\mu$, E_f; exact $\delta n(x)$ $(\ell \times x)$ (§9a)
`ramp_times`, `ramp_dQ_t_mps`, `ramp_dQ_t_exact`, `ramp_dn_t_mps`, `ramp_dn_t_exact`, `ramp_dQ_adiabatic_t`, `ramp_leak_*`, `ramp_dQ_final_*`	$\Delta Q(t)$, $\delta n(x,t)$ $(x \times t)$, adiabatic ΔQ, leakage, final ΔQ; TDVP and exact (§10a)

Part II

Dense Python notebook `msh_chain_python_ed.ipynb`

The notebook reruns the TN calculations with the dense NumPy BdG code "from the earlier work", so the two can be compared number by number. Imports: `numpy`, `matplotlib`, `numba`, `warnings` (and `ipywidgets` in §7b). All results go into the dict `results`, saved at the end. Sites are 0-indexed. Section numbers below are those of the Python notebook; Section 1.4 gives the Julia counterparts.

12 Model (Python §0)

Constants. `pauli` is the 4×2×2 array $(\sigma_0, \sigma_x, \sigma_y, \sigma_z)$. Kronecker products $\tau_a \otimes \sigma_b$ (`t` = particle–hole, `s` = spin): `tzs0`, `tzsy`, `t0sx`, `t0sy`, `t0sz`, `txs0`, `tysy`. `params = dict(t=1.0, mu=-7.0, alpha=0.9, Delta=2.4, J=5.8)`.

`smoothstep(t)`

`@numba.vectorize`: 0 for $t < 0$, 1 for $t > 1$, $\sin^2(\pi t/2)$ otherwise. Same as the Julia `smoothstep`.

`spin_texture(f, N, phi=np.pi/2)`

Returns a dict `S[x] = (Sx, Sy, Sz)` for $x = 0 \ldots N-1$ with $\theta_x = (-1)^x \frac{\pi}{2} f(x)$. Unlike the Julia closure, it takes N and evaluates all sites at once.

`calc_msh_ham(N, S, t, mu, alpha, Delta, J, pbc=False)`

Real-space BdG matrix in the spinor $\psi^\dagger = (c_\uparrow^\dagger, c_\downarrow^\dagger, c_\downarrow, -c_\uparrow)$, built as an $(N, 4, N, 4)$ array and reshaped to $4N \times 4N$ (site-major):

```
H[i,:,i,:] = -mu * tzs0 + Delta * txs0                    # onsite and s-wave pairing
H[x,:,x,:] += J * (Sx * t0sx + Sy * t0sy + Sz * t0sz)     # exchange
H[i+1,:,i,:] = -t * tzs0 - 1j*alpha * tzsy                # hopping and Rashba
H[i,:,i+1,:] = -t * tzs0 + 1j*alpha * tzsy
```

`pbc=True` adds the bond between the chain ends. In this basis the pairing is $\Delta\tau_x$ and the exchange is $\tau_0 \otimes J\,\mathbf{S}\cdot\sigma$. The Rashba sign is opposite to the Julia model (see Overview).

`msh_ham(N, f, **kw)`

`calc_msh_ham(N, spin_texture(f, N), ****params, **kw)`: the Hamiltonian for a rotation profile, with parameter overrides. Used everywhere below.

`reference_julia(N, f, t=1.0, mu=-7.0, alpha=0.9, Delta=2.4, J=5.8, phi=np.pi/2)`

Port of the Julia `reference`: basis (nambu, spin, x), hole block $-H_0^*$. Used in §1 as a check that the Python and Julia models agree.

`isPH(H)`

Returns $\max|H + (\mathbb{1}_N \otimes \tau_y\sigma_y)\, H^*\, (\mathbb{1}_N \otimes \tau_y\sigma_y)|$, the particle–hole error for $H = -\tau_y\sigma_y H^* \tau_y\sigma_y$.

`particle_hole_weights(wf)`

Spin-summed particle weights $|w_{x,0}|^2 + |w_{x,1}|^2$ and hole weights $|w_{x,2}|^2 + |w_{x,3}|^2$ per site, each $N \times n_{\text{states}}$, from the reshaped $(N, 4, n)$ wavefunctions. Python counterpart of the `wp`, `wh` lines of Julia `ldos` and of `uv_profile`.

`lorentz_ldos(en, w, omega, eta)`

$\sum_k w[x,k]\,\delta_\eta(\omega - E_k)$ with $\delta_\eta(x) = (\eta/\pi)/(x^2 + \eta^2)$, returned as $N_\omega \times N$. Together with `particle_hole_weights` it equals Julia `ldos`.

`ramp_f, ramp_duration, ramp_wall, cases_for`

```
def ramp_f(time, N, ell_final=4, rot_T=100.0, step_T=0.1, t0=0.0)
def ramp_duration(N, ell_final=4, rot_T=100.0, step_T=0.1, t0=0.0)
def ramp_wall(time, N, ell_final=4, rot_T=100.0, step_T=0.1, t0=0.0)
def cases_for(N, ell_final=0)
```

Ports of the Julia `SpinRamp` methods: `ramp_f` returns $n \mapsto f_n(t)$ (Julia `f_nt`), `ramp_duration` is `duration`, `ramp_wall` is `wall_L` (vectorized with `np.clip`). `cases_for(N, ell_final=0)` is `ldos_snapshots`: four labelled profiles at $t = 0, T/3, 2T/3, T$ of the ramp with every site rotated.

13 Hamiltonian check and spectrum sweep (Python §1–2)

§1. For the four snapshots, `calc_msh_ham` is compared with `reference_julia`. The bases differ, so the cell compares spectra and the spin-summed particle LDOS (`lorentz_ldos`, $\eta = 0.03$), both basis-independent, and prints `isPH`:

Snapshot	$\max \lvert E - E_{\text{ref}} \rvert$	LDOS error	PH error	$\lvert E \rvert_1$	$\lvert E \rvert_3$
$t = 0$	1.2e-14	1.5e-13	0	9.51e-17	0.255
$t = T/3 = 137$	8.9e-15	4.9e-13	0	2.00e-10	0.257
$t = 2T/3 = 273$	1.2e-14	8.9e-13	0	1.55e-04	0.277
$t = T = 410$	1.1e-14	9.8e-13	0	1.38	1.378

§2. The uniform-rotation sweep $f = 0, 0.1, \ldots, 1$ (`np.linalg.eigvalsh(msh_ham(N, lambda n: fv))`). The printed $|E|_1, |E|_3$ match the Julia §5 table; saved as `sweep_f`, `sweep_E13`.

14 Lorentzian LDOS (Python §3)

Same four snapshots, $\omega \in [-3, 3]$ (601 points), $\eta = 0.03$, three-row figure as Julia §6. For each snapshot: `np.linalg.eigh(msh_ham(N, f))` → `particle_hole_weights` → `lorentz_ldos` for particle and hole. Printed $\max |A_h(\omega) - A_p(-\omega)|$: 4.3e-13, 7.4e-13, 6.9e-13, 7.7e-13. The lambda `cmax` is the 99.5th-percentile colour limit. Saved: `ldos_omega`, `ldos_p_j`, `ldos_h_j`.

15 Jackson-KPM LDOS (Python §4)

Port of Julia `kpm_exact` and TensorBinding's `_kpm_weight_matrix`: the KPM LDOS from the exact eigenpairs, which is what `get_ldos_spatial` converges to as the MPS truncation error goes to zero. Results are in physical units.

`kpm_scale(t=1.0, mu=-7.0, alpha=0.9, Delta=2.4, J=5.8)`

$1.05\,(|\mu| + |J| + 2\sqrt{t^2+\alpha^2} + |\Delta|)$, the `msh_tb` default. Printed: scale = 18.785261.

`jackson(Ncheb)`

Unnormalized Jackson factors $N_{\text{cheb}}\, g_n$, "as in TensorBinding's `_kpm_kernel`":

$$N_{\text{cheb}}\, g_n = (N_{\text{cheb}} - n)\cos\frac{\pi n}{N_{\text{cheb}}} + \sin\frac{\pi n}{N_{\text{cheb}}}\cot\frac{\pi}{N_{\text{cheb}}}, \qquad n = 0 \ldots N_{\text{cheb}} - 1.$$

The textbook Jackson kernel for M moments (Weiße *et al.*, Rev. Mod. Phys. **78**, 275 (2006)) is $g_n = [(M-n+1)\cos\frac{\pi n}{M+1} + \sin\frac{\pi n}{M+1}\cot\frac{\pi}{M+1}]/(M+1)$; with $M = N_{\text{cheb}} - 1$ it is exactly the expression above divided by N_{cheb}. So using N_{cheb} in place of $N_{\text{cheb}} + 1$ is the Jackson kernel for $N_{\text{cheb}} - 1$ moments: $g_0 = 1$ (the normalization, hence the sum rule, is unchanged) and the weight of the last moment is $\cos(\pi - \pi/N) + \cos(\pi/N) = 0$. The resolution is that of $N_{\text{cheb}} - 1$ moments, a relative change of $1/N_{\text{cheb}}$. TensorBinding's `_kpm_kernel` says the same in its source comment; the code now carries this explanation.

`kpm_weight_matrix(Ncheb, w)`

$W_{n,\tilde\omega} = c_n\, N_{\text{cheb}} g_n\, T_n(\tilde\omega)$ for $|\tilde\omega| < 1$ (0 outside), with $c_0 = 1$, $c_{n>0} = 2$ and $T_n(\tilde\omega) = \cos(n \arccos\tilde\omega)$.

`kpm_ldos(en, w, omega, Ncheb, scale, center=0.0)`

Jackson-KPM LDOS in physical units, $N_\omega \times N$, from eigenvalues `en` and weights `w[x, k]`:

$$\mu_n(x) = \sum_k w[x,k]\, T_n(\tilde E_k), \qquad A(x,\omega) = \frac{\pi}{\text{scale}} \cdot \frac{\sum_n W_{n,\tilde\omega}\, \mu_n(x)}{\pi^2 N_{\text{cheb}}\sqrt{1-\tilde\omega^2}},$$

i.e. the standard KPM expansion $\frac{1}{\pi\,\text{scale}\sqrt{1-\tilde\omega^2}}[g_0\mu_0 + 2\sum_{n\ge1} g_n\mu_n T_n(\tilde\omega)]$. $\tilde E$ is clipped to $[-1,1]$ and $A = 0$ for $|\tilde\omega| \ge 1$.

§4 cell. Four snapshots, $N_{\text{cheb}} = 1024$, 241 points in $[-3,3]$, weights $p + h$. A sum-rule check integrates the LDOS over 20001 points in ± 0.999 scale: $\int A\, d\omega = 4.0000$ on every site for every snapshot (expected 4: two spins × two Nambu sectors). Printed max A: 2.594, 4.774, 4.854, 4.854. Saved: `kpm1024_omega`, `kpm1024_ph_j`.

16 Majorana modes (Python §5)

The Majorana construction of the earlier work (`takagi`, `individual_MZMs`, `fix_unitary`) with a `np.repeat` position operator. `individual_MZMs` now accepts `nzero`, which selects the `nzero` states with smallest $|E|$ (as the Julia code does) instead of the window $|E| <$ `tol`; for short segments the pair splitting exceeds any sensible `tol`.

Function	What it does
`dag(A, axis1, axis2)`	Hermitian conjugate (swap the last two axes and conjugate; conjugate only for 1D).
`PHconj(psi)`	Particle–hole conjugation $C\psi = \tau_y\sigma_y\psi^*$ applied site by site to the columns of `psi` ("`ltslts`", "`udd-u`").
`dQ_mode(psi)`	$\Delta Q = \langle\psi\|\tau_z\|\psi\rangle = \sum\|u\|^2 - \|v\|^2$ (Julia `ΔQ_mode`).
`dn_mode(psi)`	$\delta n(x) = \sum_\sigma \|u_\sigma(x)\|^2 - \|v_\sigma(x)\|^2$ (Julia `δn_mode`).

`takagi(A)`

Comment in the code: "Diagonalize a complex symmetric matrix as $A = UDU^T$, U unitary and D diagonal". Steps:

```
B = dag(A)@A;  b, V = np.linalg.eigh(B)        # eigenbasis of A†A
C = V.T@A@V;   X = C.real;  Y = C.imag
w, W = np.linalg.eigh(X+Y);  U = V@W
z = np.sum(U*(A@U), axis=0);  r = np.abs(z)     # z_k = (UᵀAU)_kk
U *= (r/z)**0.5                                 # phase fix so that the diagonal is r ≥ 0
return r, U.conj()
```

Returns the Takagi values r and the conjugated factor.

Why it works (now also in the code comments). $C = V^T A V$ is complex symmetric, and $C^\dagger C = V^\dagger A^\dagger A V = \mathrm{diag}(b)$ because $V^* V^T = (VV^\dagger)^* = \mathbb{1}$. Writing $C = X + iY$ with X, Y real symmetric, $C^\dagger C = X^2 + Y^2 + i[X, Y]$; this is real, so $[X, Y] = 0$. Commuting real symmetric matrices have a common real orthogonal eigenbasis W, which `eigh(X+Y)` finds as long as $X + Y$ has no accidental degeneracy. Then $U = VW$ is unitary and $U^T A U = W^T C W = \mathrm{diag}(z_k)$; the phase fix $U \to U\,\mathrm{diag}(e^{-i\theta_k/2})$ makes it $\mathrm{diag}(r_k)$. Hence $A = U^*\,\mathrm{diag}(r)\,U^\dagger$, i.e. the factor Z of $A = ZDZ^T$ is $Z = U^*$, which is why the function returns `U.conj()`. In the zero-mode pair $A^\dagger A = \mathbb{1}$ ($b_k = 1$, any V is an eigenbasis), and the argument still holds.

`individual_MZMs(E, M, Nx, Ny, tol=1e-3, nzero=None)`

Separates the zero-energy subspace into localized Majoranas. Returns `(mbs, mask, M, r)`: the Majoranas ($4N \times n$, sorted by position), the boolean column mask of the subspace, the eigenvector matrix and the centres $\langle x \rangle$.

1. *Subspace.* With `nzero`: the `nzero` smallest $|E|$. Otherwise $|E| <$ `tol`; if the count is odd, the next-smallest state is added with a warning.
2. *Majoranas.* $C_{ij} = \langle \psi_i | C \psi_j \rangle$ (`dag(psi) @ PHconj(psi)`) is factorized with `takagi`; $\gamma = \psi Z$ are "eigenstates of C" (code comment). Indeed, C is antiunitary and the subspace is closed under it, so $C\psi_j = \sum_i \psi_i C_{ij}$ and $C\gamma = (C\psi)Z^* = \psi\, C_{\mathrm{mat}} Z^* = \psi Z D\, Z^T Z^* = \gamma D$ with $Z^T Z^* = (Z^\dagger Z)^* = \mathbb{1}$; $D = \mathbb{1}$ here, so every column is a Majorana. This is the conjugated factor that `takagi` returns.
3. *Localize.* The site-major position operator `x = np.repeat([i*Ny + j ...], 4)` (here $N_y = 1$, so x = site index) gives $R = \gamma^\dagger x \gamma$; `np.linalg.eigh(R.real)` returns ascending centres r and the rotation O; `mbs = gamma @ O`. Since C commutes with x, $R_{ij} = \langle \gamma_i | x \gamma_j \rangle = \langle C\gamma_i | x C \gamma_j \rangle^* = R_{ij}^*$ is real (`.real` only drops round-off); O is then real orthogonal and γO stays self-conjugate.

`fix_unitary(mbs, mask, M)`

Pairs Majoranas 0–1, 2–3, ... into fermions with $U = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ i & -i \end{pmatrix}$, i.e. columns $(\gamma_{2k} + i\gamma_{2k+1})/\sqrt{2}$ and $(\gamma_{2k} - i\gamma_{2k+1})/\sqrt{2}$, reorders them (all first members, then all second members, with the first half reversed) and writes them into `M[:, mask]`. With $\varphi_k = (\gamma_{2k} + i\gamma_{2k+1})/\sqrt{2}$, the second column of each pair is $C\varphi_k$. The first reordering gives $[\varphi_0, \varphi_1, \ldots, C\varphi_0, C\varphi_1, \ldots]$; reversing the first half gives $[\varphi_{n/2-1}, \ldots, \varphi_0, C\varphi_0, \ldots, C\varphi_{n/2-1}]$, so that column $2N - 1 - k$ holds φ_k and column $2N + k$ holds $C\varphi_k$. That is the structure of the `eigh` columns of any BdG matrix: with energies sorted, $E_{2N+k} = -E_{2N-1-k}$ and the two columns are C partners. The fixed M is therefore again of the form $\begin{pmatrix} U & V^* \\ V & U^* \end{pmatrix}$ up to the column order, and the masks (column $2N+k$ occupied $\Leftrightarrow$ column $2N-1-k$ empty) work for any number of pairs. For $n = 2$ the reversal does nothing (the reason for it is now in the code comment). For $n = 2$ the result is column $2N - 1 = (\gamma_1 + i\gamma_2)/\sqrt{2}$, column $2N = (\gamma_1 - i\gamma_2)/\sqrt{2}$, which `majorana_fermion` may then swap.

`majorana_fermion(H)`

```
"""Majoranas γ1, γ2 and fermion ψf (+E member) of the lowest pair.

Returns centers, γs (4N x 2), ψf, E_f and the fixed unitary M (column 2N = ψf, 2N-1 = Cψf).
"""
```

Full diagonalization (`np.linalg.eigh`), `individual_MZMs(..., nzero=2)`, `fix_unitary`, then the energies $\mathrm{Re}\langle M_k|H|M_k\rangle$ of the two pair columns; if column $2N$ has negative energy, the two columns are swapped so that column $2N$ is always the $+E$ member ψ_f. Returns `dict(centers, gammas, psi_f, E_f, M, en)`. The fixed unitary `M` (all $4N$ eigenvectors with the pair replaced) is what the ramp evolves. Julia counterpart: `majorana_fermion(HB, sites)` (DMRG instead of full diagonalization).

§5 cell. The same two textures as Julia §8a. Printed: centres [1.41, 61.59] and [16.55, 46.45]; $\|C\gamma_i - \gamma_i\| \le 2.8\times10^{-14}$; $\max|\langle\gamma_i|\gamma_j\rangle - \delta_{ij}| \le 1.4\times10^{-15}$; $\max|\mathrm{Re}\,H_{ij}| \le 5.1\times10^{-17}$; all normalizations 1 to 12 digits; $E_f = 1.179\times10^{-14}$ (exact 5.070×10^{-15}) and 3.319×10^{-8} (exact 3.319×10^{-8}). Saved: `mzm_centers_j`, `mzm_E_f_j`, `mzm_gamma_weight_j`.

17 Charge difference vs Majorana separation (Python §6)

Centred topological segment of ℓ sites. Three values per ℓ: the single-mode $\Delta Q = \langle\psi_f|\tau_z|\psi_f\rangle$ (what Julia `majorana_pair_charge` computes); the many-body $Q(|1\rangle) - Q(|0\rangle)$ from the charge density of Slater states built from the fixed unitary; and the Feynman–Hellmann $-\partial E_f/\partial\mu$.

Function	What it does
`segment(N, ell)`	`range((N-ell)//2, (N-ell)//2 + ell)`, same as Julia.
`f_segment(N, ell)`	0 inside the segment, 1 outside.
`charge_density(M, cols)`	Spin-summed electron density per site of the state filling columns `cols`: particle diagonal of $G = M_{\mathrm{cols}}M_{\mathrm{cols}}^\dagger$, $\rho(x) = G_{4x,4x} + G_{4x+1,4x+1}$.

`pair_charge(L, ell, dmu=1e-4, **kw)`

Python counterpart of `majorana_pair_charge` and `exact_pair_charge` combined. Builds `msh_ham(N, f_segment(N, ell))`, calls `majorana_fermion` and uses the fixed unitary `M`:

```
occ0 = np.arange(2*N)                                # |0⟩: columns 0 … 2N−1
occ1 = np.r_[np.arange(2*N - 1), 2*N]                # |1⟩: 2N−1 → 2N
drho = charge_density(M, occ1) - charge_density(M, occ0)
```

$|0\rangle$ fills columns $0\ldots2N-1$ (negative-energy bulk and $C\psi_f$); $|1\rangle$ replaces column $2N-1$ by column $2N$ (ψ_f). The Feynman–Hellmann value uses the smallest $|E|$ at $\mu \pm d\mu$ (central difference). Returns `dict(dQ, dQ_mb, dQ_FH, E, dn, drho)`: single-mode ΔQ (`dQ_mode`), many-body $\sum_x \Delta\rho$, $-\partial E_f/\partial\mu$, E_f, δn (`dn_mode`) and $\Delta\rho(x)$.

§6 cell. $\ell = 2, 4, \ldots, 48$ on 64 sites. Single-mode and many-body values agree to the printed digits at every ℓ (e.g. $\ell = 2$: $+0.7214509$; $\ell = 48$: $+1.496954\times10^{-9}$ vs $+1.496953\times10^{-9}$), $\max|\delta n - \Delta\rho| \le 2.1\times10^{-15}$, and $-\partial E/\partial\mu$ agrees with them to about 10^{-3} relative or better; the largest relative differences are at $\ell \ge 44$ (e.g. $\ell = 44$: 9.985×10^{-9} vs 9.994×10^{-9}). E_f falls from 0.147 ($\ell = 2$) to 1.19×10^{-11} ($\ell = 48$). Saved: `charge_ells`, `charge_dQ`, `charge_dQ_mb`, `charge_dQ_FH`, `charge_E`, `charge_dn`.

18 Spin ramp (Python §7)

Same two-sided ramp as Julia `SpinRamp` (`ramp_f`). The notebook notes that the earlier `spin_timedep` rotated from the right end only.

`run_spin_ramp(L, dt=0.1, nsnap=200, model_kw={}, **paramsT)`

Initial state. `majorana_fermion` of $H(0)$; the whole $4N \times 4N$ fixed unitary `M` is evolved (column $2N = \psi_f$, column $2N-1 = C\psi_f$).

Logical states as masks (as in the CZ, QFT and period-finding notebooks):

```
masks = np.zeros((n_states, 4*N), bool)       # columns 2N+1 … 4N−1 (excited states) empty
masks[1, 2*N] = True                          # |1⟩: fermion mode ψf filled; |0⟩: empty
masks[:, :2*N] = np.flip(~masks[:, 2*N:], axis=1)   # particle-hole partners: column 2N−1−k =
 ↪  C(column 2N+k)
```

So $|0\rangle$ fills columns $0 \ldots 2N-1$, and $|1\rangle$ fills $0 \ldots 2N-2$ and $2N$.

Time step. With H at the midpoint $t + dt/2$ (as the Julia code does),

$$\psi \leftarrow V\, e^{-iE\,dt}\, V^\dagger \psi, \qquad (E, V) = \texttt{eigh}(H(t + dt/2)),$$

followed by normalization of every column. $n_{\text{steps}} = \text{round}(T/dt)$, $dt = T/n_{\text{steps}}$.

Observables at the samples (the same 201 indices as Julia, `round(linspace(1, nsteps+1, nsnap+1)) - 1`): for each logical state j, $G_j = \psi[:, \text{mask}_j]\, \psi[:, \text{mask}_j]^\dagger$ and the per-spin particle diagonal `charge_density_t[n, j, x, σ]`; then

$$\delta n(x,t) = \sum_\sigma (\rho_1 - \rho_0)(x, \sigma, t), \qquad \Delta Q(t) = \sum_x \delta n(x,t).$$

Since both states share the bulk columns and all columns evolve with the same $U(t)$, $G_1 - G_0 = |\psi_f\rangle\langle\psi_f| - |C\psi_f\rangle\langle C\psi_f|$ at every time. The single-mode values $\langle\psi_f|\tau_z|\psi_f\rangle$ and $|u|^2 - |v|^2$ of column $2N$ (what Julia evolves) are recorded as a check, together with $W = |\psi_f(x,t)|^2$.

End of ramp. Column $2N$ is projected on the instantaneous pair of $H(T)$ (first positive eigenvalue and the one below): leakage $= 1 - p_+ - p_-$. If it ends in the $-E$ member, $|0\rangle$ and $|1\rangle$ swap and the sign of ΔQ flips (`sgn`).

Returns `T, dt, E0, leak, p_wrong, dQ_final, dQ_adiabatic` (ΔQ of the $+E$ eigenvector at T), `times, dQ_t, dn_t, charge_density_t, W_t, dQ_t_single, dn_t_single`.

§7 cell: two Rashba signs. The ramp is run with `ramp_params = dict(ell_final=4, rot_T=100.0, step_T=0.1)` ($T = 390$) for $\alpha = +0.9$ ("paper", this notebook's sign) and $\alpha = -0.9$ ("julia", the model of `msh_chain_tn`). Static quantities do not depend on the sign, but the dynamics do; only the $\alpha = -0.9$ run is expected to match Julia, and only it is plotted.

	$\alpha = +0.9$ (paper)	$\alpha = -0.9$ (julia)
$E(0)$	1.2e-14	1.2e-14
leakage	2.179e-04	2.143e-04
wrong branch	1.1e-27	1.1e-26
$\Delta Q(T)$	+0.523281	+0.516615
adiabatic $\Delta Q(T)$	+0.519062	+0.519062
$\max_t \lvert\Delta Q - \langle\psi_f\lvert\tau_z\rvert\psi_f\rangle\rvert$	2.8e-14	2.0e-14
$\max \lvert\delta n - (\lvert u\rvert^2 - \lvert v\rvert^2)\rvert$	1.5e-14	1.2e-14
$\max_t \lvert\Delta Q_{\text{paper}}(t) - \Delta Q_{\text{julia}}(t)\rvert$	2.4e-02	

The $\alpha = -0.9$ numbers equal the Julia exact values (leak 2.143×10^{-4}, $\Delta Q(T) = +0.516615$). Saved: `ramp_{paper,julia}_{times, dQ_t, dn_t, leak, p_wrong, dQ_final, dQ_adiabatic}`.

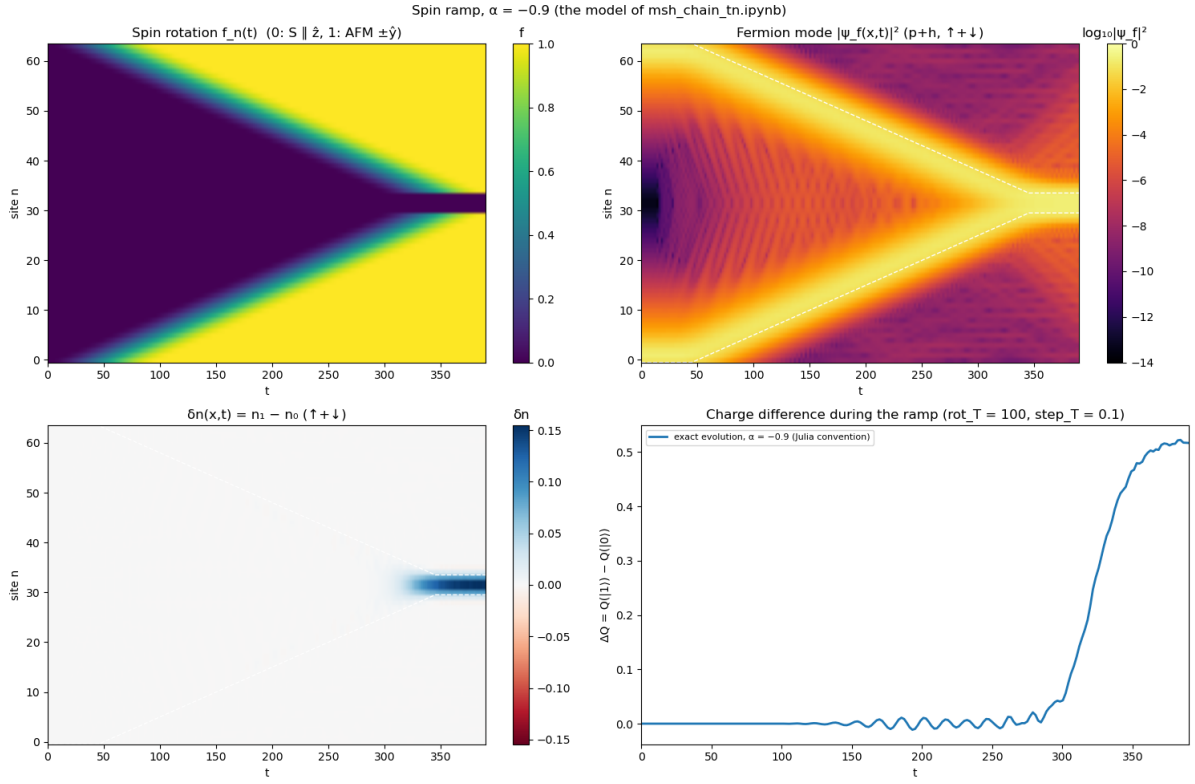


Figure 6: Python §7, $\alpha = -0.9$: rotation profile, $|\psi_f(x,t)|^2$ (column $2N$, log scale, dashed wall centres), $\delta n(x,t)$ from the masks, and $\Delta Q(t)$. Compare Fig. 5.

19 Interactive views and saving (Python §7b, last cell)

`plot_spins(l=0)`

`@interact` slider over `ts = np.linspace(0, T, 201)` ($N = 64$, `ramp_params`). Draws the spin texture at `ts[l]` as arrows (S_x, S_y components, colour from $(1 + \mathbf{S})/2$), prints the 12 eigenvalues around zero, and shows the particle and particle + hole LDOS ($\eta = 10^{-2}$, $\omega \in [-2, 2]$) of `msh_ham`.

`plot_zero_energy_ldos(l=0)`

Slider as above; plots the particle and particle + hole LDOS at $\omega = 0$ ($\eta = 10^{-2}$) vs site at `ts[l]`.

Save cell. `np.savez("msh_compare_python.npz", ...)` writes every entry of `results`. The file contains 42 arrays, the keys listed in the sections above (output: "saved 42 arrays").

Part III

Comparison notebook `msh_chain_compare.ipynb`

20 Method

The notebook loads `tn = np.load("msh_chain_tn_results.npz")` and `py = np.load("msh_compare_python.npz")` and compares them row by row. It runs in seconds.

`check(section, name, a, b, tol)`

```
d = float(np.max(np.abs(np.asarray(a) - np.asarray(b))))
rows.append((section, name, d, tol))
print(f"{name:<58s} {d:9.1e}  (tol {tol:.0e})  {'✓' if d <= tol else '✗'}")
```

Computes $d = \max |a_{\text{Julia}} - b_{\text{Python}}|$, stores it in `rows` with its category, and prints ✓ if $d \leq$ `tol`. `section` is `"dense"` or `"tn"`.

Tolerance classes. The notebook states the rule: dense-against-dense rows (spectra, Lorentzian LDOS, exact KPM, exact ΔQ and δn, exact ramp) should agree to round-off and use 10^{-9}; the finite-difference $-\partial E_f/\partial\mu$ uses 10^{-8}. Tensor-network-against-dense rows (`get_ldos_spatial`, DMRG Majoranas, MPS ΔQ, TDVP) agree to the truncation error of the method and use 10^{-4} (LDOS, Majorana centres, TDVP ΔQ and δn) or 10^{-6} (Majorana profiles, MPS $\Delta Q(\ell)$, TDVP leakage); E_f of the DMRG Majoranas is held to 10^{-9}. Only basis-independent quantities are compared, and the ramp rows use the Python run with $\alpha = -0.9$ (`ramp_julia_*`). The arrays `charge_ells` and the ramp times are asserted equal before comparison.

Tensor-network tolerances, row by row. The summary markdown of the notebook (§6) sets each tensor-network tolerance against the error source of that method, at the size it would have in an acceptable but unconverged run:

Row	Tol.	Error source and scale	This run
`get_ldos_spatial`	10^{-4}	MPO Chebyshev recursion truncated at each of 1024 steps (`maxdim = 512`, `cutoff = 1e-12`); grows with the entanglement of H (largest at $t = T$). $10^{-4} \approx 2 \times 10^{-5}$ of the LDOS peak (≈ 4.9).	3.4×10^{-6}–3.4×10^{-5}
Majorana centres	10^{-4}	$\langle x \rangle$ is linear in the DMRG weight error ε with lever arm up to 64 sites; 10^{-4} site $\leftrightarrow$ $\varepsilon \sim 10^{-6}$.	2×10^{-9}, 1.3×10^{-8}
$\lvert\gamma\rvert^2$ profiles	10^{-6}	DMRG on K leaves a bulk admixture of amplitude 10^{-7}–10^{-6} (§8); weights err at most by that order.	3×10^{-10}, 4×10^{-9}
E_f	10^{-9}	Quadratic in the state error ($\lesssim 10^{-12}$ for a 10^{-6} error); the dense tolerance.	2×10^{-15}, 3.5×10^{-15}
MPS $\Delta Q(\ell)$	10^{-6}	Linear in the bulk admixture; DMRG without refinement is off by up to 1.5×10^{-6}, so this asks for the refined accuracy.	2.3×10^{-8}
TDVP ΔQ, δn	10^{-4}	Same midpoint propagator on both sides, so the time-step error cancels; remaining TDVP projection/Krylov error (tol 10^{-12}, 3900 steps, `maxdim = 64`) and the DMRG error of $\psi_f(0)$, against $\lvert\Delta Q\rvert \leq 0.52$, $\lvert\delta n\rvert \leq 0.15$.	4.1×10^{-7}, 2.6×10^{-7}
TDVP leakage	10^{-6}	The leakage is 2.1×10^{-4}; this asks for it to better than 0.5 %.	2.9×10^{-9}

The summary cell also prints the row closest to its tolerance in each class.

21 What each block compares

1. **Spectrum and Lorentzian LDOS** (Julia §5–6 / Python §2–3): $|E|_1, |E|_3$ of the f sweep; particle and hole LDOS ($\eta = 0.03$) of the four snapshots, stacked horizontally.
2. **KPM LDOS** (Julia §7a / Python §4): `kpm1024_exact_j` vs `kpm1024_ph_j` tests the KPM formula and normalization; `kpm1024_tb_j` vs `kpm1024_ph_j` tests the tensor-network arithmetic of `get_ldos_spatial`.
3. **Majorana modes** (Julia §8a DMRG / Python §5 Takagi): centres, $|\gamma_{1,2}(n)|^2$ profiles (p + h, ↑ + ↓), E_f, for $f = 0$ and the domain texture.
4. **Charge difference** (Julia §9a / Python §6): Julia `exact_pair_charge` against Python single-mode and many-body ΔQ; $-\partial E_f/\partial\mu$ (both $d\mu = 10^{-4}$); $E_f(\ell)$; $\delta n(x)$ for every ℓ; and Julia's final MPS value (refined where $E_f < 2 \times 10^{-6}$) against Python.
5. **Spin ramp** (Julia §10a / Python §7): $\ell_{\text{final}} = 4$, $T = 390$, 201 samples. Julia's dense evolution against Python (round-off), and TDVP against Python (truncation).

22 Results

All 32 comparisons are within tolerance (printed: "32 of 32 comparisons within tolerance"). Closest to tolerance: among the dense rows the Lorentzian LDOS at $t = 2T/3$ (1.2×10^{-11}, 1.2×10^{-2} of its tolerance), among the tensor-network rows `get_ldos_spatial` at $t = T$ (3.4×10^{-5}, 0.34 of its tolerance).

Table 8: Comparison rows as printed by `msh_chain_compare.ipynb`: $d = \max|\text{Julia} - \text{Python}|$.

Row	Class	d	tol
f sweep $\|E\|_1, \|E\|_3$	dense	2.8e-14	1e-09
Lorentzian LDOS particle, hole: $t = 0$	dense	8.7e-13	1e-09
Lorentzian LDOS particle, hole: $t = T/3$	dense	7.5e-12	1e-09
Lorentzian LDOS particle, hole: $t = 2T/3$	dense	1.2e-11	1e-09
Lorentzian LDOS particle, hole: $t = T$	dense	3.8e-12	1e-09
kpm_exact (Julia) vs Python, p+h: $t = 0$	dense	3.8e-13	1e-09
kpm_exact (Julia) vs Python, p+h: $t = T/3$	dense	3.4e-12	1e-09
kpm_exact (Julia) vs Python, p+h: $t = 2T/3$	dense	9.0e-12	1e-09
kpm_exact (Julia) vs Python, p+h: $t = T$	dense	1.8e-12	1e-09
get_ldos_spatial, $N_{\text{cheb}} = 1024$, p+h: $t = 0$	tn	5.3e-06	1e-04
get_ldos_spatial, $N_{\text{cheb}} = 1024$, p+h: $t = T/3$	tn	3.8e-06	1e-04
get_ldos_spatial, $N_{\text{cheb}} = 1024$, p+h: $t = 2T/3$	tn	3.6e-06	1e-04
get_ldos_spatial, $N_{\text{cheb}} = 1024$, p+h: $t = T$	tn	3.4e-05	1e-04
Majorana centres: $f = 0$	tn	2.0e-09	1e-04
$\|\gamma_1\|^2, \|\gamma_2\|^2$ profiles: $f = 0$	tn	3.3e-10	1e-06
E_f: $f = 0$	tn	2.0e-15	1e-09
Majorana centres: domain	tn	1.3e-08	1e-04
$\|\gamma_1\|^2, \|\gamma_2\|^2$ profiles: domain	tn	3.7e-09	1e-06
E_f: domain	tn	3.5e-15	1e-09
exact $\Delta Q(\ell)$: Julia vs Python single mode	dense	2.4e-14	1e-09

Row	Class	d	tol
exact $\Delta Q(\ell)$: Julia vs Python many-body (masks)	dense	2.7e-14	1e-09
$-\partial E_f/\partial\mu$ (finite difference)	dense	4.7e-11	1e-08
$E_f(\ell)$	dense	1.1e-14	1e-09
exact $\delta n(x)$ for every ℓ	dense	7.1e-15	1e-09
MPS $\Delta Q(\ell)$	tn	2.3e-08	1e-06
exact $\Delta Q(t)$	dense	3.7e-14	1e-09
exact $\delta n(x,t)$	dense	2.0e-14	1e-09
exact leakage	dense	1.0e-14	1e-09
adiabatic ΔQ at $t=T$	dense	1.3e-14	1e-09
MPS / TDVP $\Delta Q(t)$	tn	4.1e-07	1e-04
MPS / TDVP $\delta n(x,t)$	tn	2.6e-07	1e-04
MPS / TDVP leakage	tn	2.9e-09	1e-06

The dense rows agree at 10^{-15}–10^{-11}; the largest dense difference is the finite-difference $-\partial E_f/\partial\mu$ (4.7×10^{-11}). The tensor-network rows sit two orders of magnitude or more below their tolerances, except `get_ldos_spatial` at $t=T$ (3.4×10^{-5} against 10^{-4}).

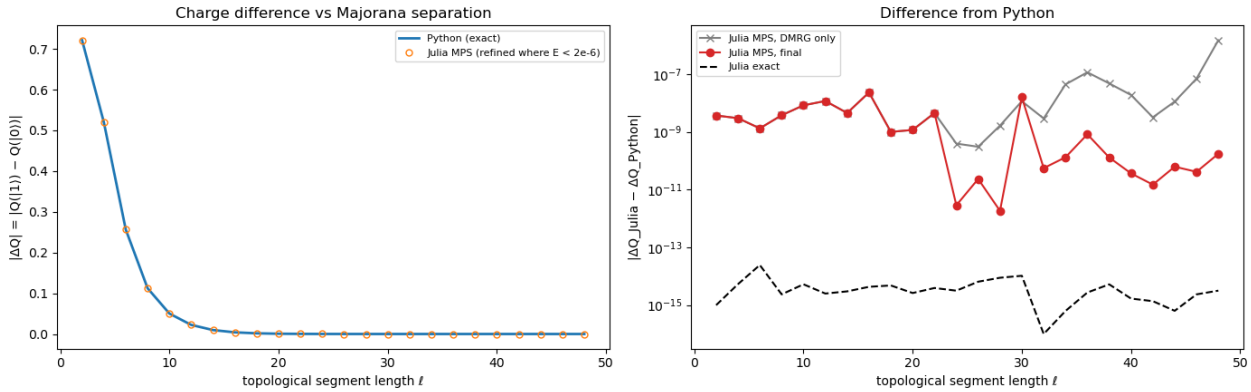


Figure 7: Compare §4. Left: $|\Delta Q|$ vs ℓ, Python exact and Julia MPS. Right: $|\Delta Q_{\text{Julia}} - \Delta Q_{\text{Python}}|$ for the MPS value with and without refinement and for Julia's exact value (round-off, $\lesssim 10^{-13}$).

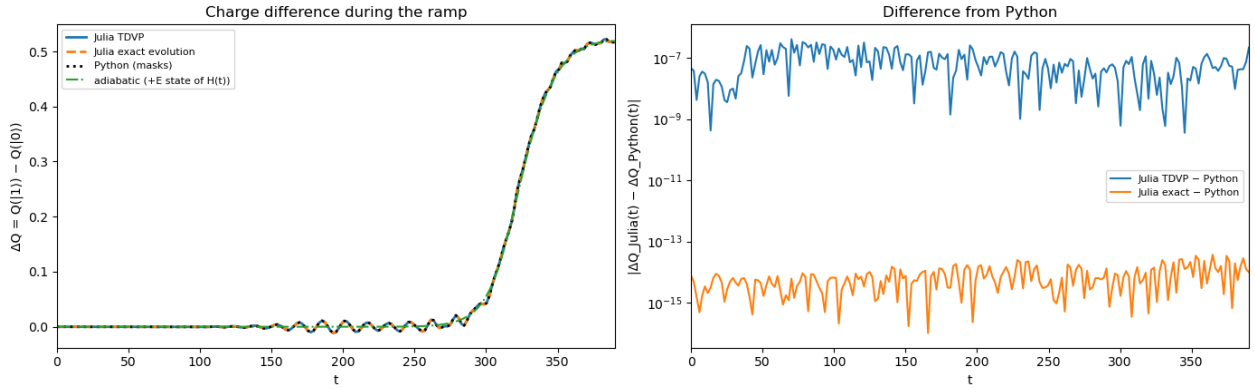


Figure 8: Compare §5. Left: $\Delta Q(t)$ from Julia TDVP, Julia exact evolution, Python (masks), and the adiabatic value. Right: differences from Python: TDVP up to 4.1×10^{-7}, Julia exact up to 3.7×10^{-14}.

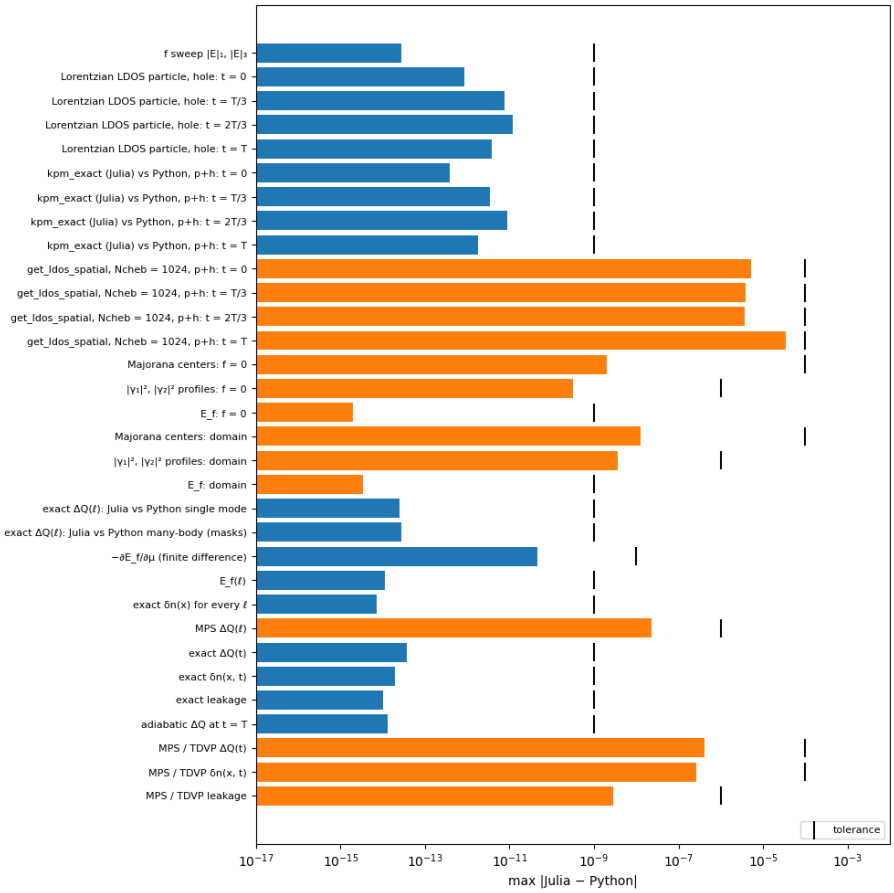


Figure 9: Compare §6 summary: max |Julia − Python| per row (blue: dense against dense; orange: tensor network against dense), with each row's tolerance (black tick).

23 TensorBinding routines used (from the v0.2.0 source)

The notebooks call several TensorBinding routines whose code they do not show. Their behaviour, read from the installed package source (TensorBinding 0.2.0), is:

`build_K(H, sites, ω, η; maxdim_K=200, cutoff_K=1e-8)` (`solvers/DMRG.jl`) returns $K = (H - \omega)^2 + \eta^2$: the MPO sum $H - \omega\mathbb{1}$ (cutoff `cutoff_K`), its square by `apply` (cutoff `cutoff_K`, bond dimension `maxdim_K`), and the sum with $\eta^2\mathbb{1}$, again truncated at `cutoff_K`. That last truncation is why the default 10^{-8} drops part of the η^2 term and the notebook passes 10^{-14} (§8).
`shift_pair_mpos(sites, q; cyclic=false)` (`core/Utils.jl`) returns $(K, K^\dagger)$ with K the binary adder $|n\rangle \to |n+q\rangle$ on the position qubits (first site most significant), a carry MPO of bond dimension 2. With `cyclic = false` the states with $n + q \geq 2^L$ are mapped to zero, so no bond joins the chain ends; this is the open-boundary Rashba term of `msh_chain` (§3).
`prepend_spin`, `prepend_nambu` (`core/AuxDOF.jl`, `core/Utils.jl`) both call `prepend_op(H, s, op)`: a new first site s carrying the 2×2 matrix `op`, joined to the first tensor of H by a bond of dimension 1, i.e. the MPO of op $\otimes H$. Symbols are looked up in fixed tables (spin: `:iSy` $= i\sigma_y$, `:Sx` $= \sigma_x/2$, ...; Nambu: `:Pp` $= |p\rangle\langle p|$, `:Ph` $= |h\rangle\langle h|$, `:tp` $= |p\rangle\langle h|$, `:tm` $= |h\rangle\langle p|$); a matrix is used as given (the notebooks pass $\sigma_{x,y,z}$ and $-\sigma^*$).
`_kpm_kernel`, `_kpm_weight_matrix(Ncheb, ω̃; kernel=:jackson)` (`solvers/kpm/kernels.jl`) return the unnormalized kernel Ng_n of the Python `jackson` (the Jackson kernel for $N - 1$ moments, see that entry) and $W_{n\tilde{\omega}} = c_n N g_n T_n(\tilde{\omega})$, zero for $|\tilde{\omega}| \geq 1$.
`get_ldos_spatial(H, Ncheb, ω; mode=:mpo, …)` (`solvers/kpm/ldos.jl`) runs the Chebyshev recursion $T_{n+1} = 2\tilde{H}T_n - T_{n-1}$ on MPOs, $\tilde{H} = (H - \text{center})/\text{scale}$, keeping three MPOs at a time and truncating each step at `maxdim`/`cutoff`. `msh_tb` sets `scale` and `center`, so no DMRG estimate

of the bounds is run. At each order T_n is projected on the selected Nambu and spin sectors (with the selectors at `nothing`, all sectors are summed) and its diagonal is read at every position with position-basis MPS, giving the moments $\mu_n(x)$; the LDOS is $A = W^T\mu/(\pi^2 N\sqrt{1-\tilde{\omega}^2})$ in package units. `kpm_exact` applies exactly this formula to the exact eigenpairs, and the two agree to 3.4×10^{-5} (compare notebook).

`evolve_with_tdvp_timedep(Hoft, ψ0, nsteps, dt; …)` (`solvers/Timeev.jl`) for step k evaluates H once at $t_{\mathrm{mid}} = (k-1)dt + dt/2$ and makes one `ITensorMPS.tdvp` call with $-iH(t_{\mathrm{mid}})$ over dt (two-site, `maxdim`, `cutoff`, normalization after each step, `reverse_step`, Krylov exponentiation with `tol`, `krylovdim`). It returns all $n_{\mathrm{steps}}+1$ states. This is the "midpoint rule" of the `run_spin_ramp` docstring; `run_spin_ramp_exact` uses the same frozen-midpoint propagator with the dense matrix exponential, which is why the time-step error cancels in the comparison.